

LLM & GenAI Engineer

The memorize-cold sheet: what you must be able to answer in your sleep

Built on the Achilles curriculum (modules 00–10, labs 01–18) and extended across the field. Interview-style Q&A first, derivations only where the answer is the derivation. Numbers are for bf16 on an H100 unless stated. Snapshot: October 2026.

How to use this sheet

- ▶ **Three passes.** Day 1: read a section. Day 3: cover every answer and say it out loud. Day 7 and 21: only the questions you missed. Spaced repetition beats re-reading.
- ▶ **Boxes.** **Blue** = interview Q&A. **Rust** = numbers and formulas to memorize. **Green** = traps interviewers use to separate memorizers from people who can do the arithmetic.
- ▶ **The four numbers of any model card:** d (width), L (layers), H/H_{kv} (query/KV heads), V (vocab). Parameters, FLOPs, KV cache and communication all follow.
- ▶ **Answer shape** for any “why” question: mechanism $\rightarrow$ number $\rightarrow$ trade-off $\rightarrow$ what you would measure.
- ▶ Pair each section with its Achilles lab ([lab NN]) — understanding shows up in what you build and measure.

THE 12 NUMBERS EVERYONE ASKS

$2mkn$	FLOPs of an $(m \times k)(k \times n)$ matmul
$2N / 6N$	forward / training FLOPs per token ($8N$ with checkpointing)
16 B/param	mixed-precision AdamW training state
≈ 295	H100 bf16 ridge point, FLOP/byte (989 TF/s / 3.35 TB/s)
$\ln V$	loss at init (10.82 for $V=50,257$; 11.76 for 128k)
$2LH_{kv}d_hb$	KV bytes per token (Llama-3-8B: 128 KiB)
$1/\sqrt{d_h}$	keeps attention logits unit-variance at init
$8d/3$	SwiGLU hidden size matching a $4d$ GELU MLP
≈ 20	Chinchilla tokens/param (compute-optimal, training only)
$2\frac{n-1}{n}S$	ring all-reduce bytes per rank
$\sqrt{p(1-p)/n}$	SE of an accuracy; $p=0.7, n=1000 \Rightarrow \pm 2.8$ pts
$0.95^{20} = 0.36$	a 95%-per-step agent over 20 steps

Contents

1 Math you actually use	1
2 ML and DL foundations interviewers still ask	2
3 Compute and hardware	3
4 GPU and TPU programming	3
5 Deep learning that matters at scale	4
6 Tokenization	5
7 The Transformer, piece by piece	6
8 Pretraining: data, scaling, parallelism	7
9 Post-training: SFT, PEFT, RLHF, DPO, GRPO	8
10 Reasoning models, post-training data, adaptation	9
11 Inference and serving	10
12 Evaluation: measure like a scientist	12
13 Interpretability, alignment, security	12
14 Applied LLM systems: RAG, agents, LLMOps	13
15 Shipping GenAI in production	14
16 Generative AI beyond text: images, multimodal	15
17 Model timeline and the tooling ecosystem	16
18 System design: answer templates	17
19 Implement from scratch: code you write blind	18
20 Worked napkin problems	18
21 The interview loop: rounds, stories, DSA	19
22 “What would you do?” debugging scenarios	20
23 Rapid fire: answer in one breath	21

24 Formula sheet: write each from memory	22
25 Paper cards: what to say in 20 seconds	22
26 Glossary A–Z	22
27 References: the canon, by section	23
28 Four-week plan and the cold self-check	24

1 Math you actually use

[module 01 · lab 01 autograd]

IDENTITIES

Matmul VJP	$Y=XW: \bar{X}=\bar{Y}W^T, \bar{W}=X^T\bar{Y}$
Softmax Jacobian	$J=\text{diag}(y)-yy^T$; VJP $\bar{z}=y\odot(\bar{y}-\langle\bar{y},y\rangle)$
Softmax CE	$\bar{z}=\text{softmax}(z)-\text{onehot}(t)$ ($\div N$ for batch mean)
LayerNorm VJP	$\bar{x}=\frac{1}{\sigma}(\bar{y}-\bar{y}-\hat{x}\overline{y\odot\hat{x}})$
RMSNorm VJP	$\bar{x}=\frac{1}{r}(\bar{y}-\hat{x}\overline{y\odot\hat{x}})$
gather / sum	VJP is scatter-add / broadcast
MLE	$\mathbb{E}_p[-\log q]=H(p)+\text{KL}(p\ q)$
Log-derivative	$\nabla\mathbb{E}_{p_\theta}[f]=\mathbb{E}[f\nabla\log p_\theta]$
Baseline	$\mathbb{E}[b\nabla\log p_\theta]=b\nabla\sum p_\theta=0$
Importance sampling	$\mathbb{E}_p[f]=\mathbb{E}_q[f p/q]$
Bradley-Terry	$P(a \succ b)=\sigma(r_a-r_b)$
KL est. ($\rho=p/q, x\sim q$)	$k_1=-\log\rho; k_3=\rho-1-\log\rho\geq 0$
SE of accuracy	$\sqrt{p(1-p)/n}$; 95% CI = ± 1.96 SE
GD on quadratic	stable iff $\eta < 2/L$; best $\eta = \frac{2}{L+\mu}$, rate $\frac{\kappa-1}{\kappa+1}$

INTERVIEW Q&A | CALCULUS AND LINEAR ALGEBRA

Q1. Derive $\partial L/\partial z$ for softmax cross-entropy.

$L = -z_t + \log \sum_k e^{z_k}$, so $\partial L/\partial z_j = -\delta_{jt} + e^{z_j}/\sum_k e^{z_k}$, i.e. $\bar{z} = \text{softmax}(z) - \text{onehot}(t)$. It sums to 0. Frameworks fuse `log_softmax` with NLL for exactly this gradient and for numerical stability (log-sum-exp with the max subtracted).

Q2. Does backprop compute the Jacobian?

No: it computes vector-Jacobian products. The Jacobian of a 4096² matmul output w.r.t. one input has $\sim 2.8 \times 10^{14}$ entries. Reverse mode costs a small constant ($\approx 2\times$) of the forward regardless of parameter count; forward mode costs one pass per input direction.

Q3. Derive the matmul VJP.

Trace trick: $dL = \text{tr}(\bar{Y}^T(dXW + XdW)) = \text{tr}((\bar{Y}W^T)^T dX) + \text{tr}((X^T\bar{Y})^T dW)$. Shape check: the gradient has the shape of the variable, and there is only one way to combine the factors into it.

Q4. Why do the rows of the softmax Jacobian sum to zero?

Adding a constant to all logits does not change the softmax, so $J\mathbf{1}=0$. A saturated (one-hot) softmax has $J\approx 0$: no gradient flows. That is why attention scales logits by $1/\sqrt{d_h}$.

Q5. Two free unit tests for a LayerNorm backward?

$\sum_j \bar{x}_j = 0$ (shift invariance) and $\sum_j \bar{x}_j \hat{x}_j = 0$ when $\epsilon = 0$ (scale invariance). RMSNorm keeps only the second.

Q6. Why can FlashAttention’s backward skip the $T \times T$ matrix $\bar{P}$?

The softmax VJP needs $\langle \bar{P}_i, P_i \rangle$ per row, which equals $\langle \bar{O}_i, O_i \rangle$ (swap sums, $\sum_j P_{ij}V_j = O_i$): one length- d dot product per row (D_i in FA-2).

Q7. What does SVD buy you in LLM work?

Eckart-Young: the best rank- r approximation keeps the top r singular values (error $\sqrt{\sum_{i>r} \sigma_i^2}$). LoRA bets updates are low rank; Muon replaces $G = U\Sigma V^T$ by UV^T ; spectral norm $\sigma_{\max}$ bounds how much a layer stretches inputs.

Q8. Two random unit vectors in $d=4096$: expected dot product?

Mean 0, variance $1/d$, std $1/64 \approx 0.016$: nearly orthogonal. High-dimensional space holds exponentially many almost-orthogonal directions $\rightarrow$ the geometric basis of superposition.

Q9. How do you gradient-check correctly?

Central differences in float64 with $\epsilon \approx u^{1/3} \approx 10^{-5}$, using a random upstream gradient G on $\sum f(x) \odot G$. In float32 you cannot tell a

subtle bug from rounding.

INTERVIEW Q&A | PROBABILITY AND INFORMATION

Q10. Why is pretraining “just” minimizing KL?

Cross-entropy = $H(p_{\text{data}}) + \text{KL}(p_{\text{data}} \| q_{\theta})$ and H is constant, so MLE minimizes forward KL. Loss is in nats/token; divide by $\ln 2$ for bits; compare across tokenizers with *bits per byte*.

Q11. Forward vs reverse KL?

Forward $\text{KL}(p \| q)$ is **mode-covering** (punishes $q \approx 0$ where $p > 0$): SFT, pretraining, classic distillation. Reverse $\text{KL}(q \| p)$ is **mode-seeking**: RLHF’s penalty $\text{KL}(\pi \| \pi_{\text{ref}})$, on-policy distillation. Example $p = (.5, .5), q = (.9, .1)$: forward 0.511, reverse 0.368 nats. KL is not a distance (asymmetric, no triangle inequality).

Q12. What is the log-derivative (REINFORCE) trick and why does RL need it?

$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}} [f(x)] = \mathbb{E} [f(x) \nabla_{\theta} \log p_{\theta}(x)]$. It needs only samples and $\log p$, not a differentiable reward — so rewards can be unit tests, verifiers or humans.

Q13. Does a baseline bias the policy gradient?

No — it reduces *variance*. $\mathbb{E} [b \nabla \log p_{\theta}] = 0$ for any b independent of the sampled x . RLOO uses the mean of the *other* samples; GRPO’s group mean includes the sample itself, which only rescales the gradient by $(1 - 1/G)$.

Q14. Where does importance sampling show up in LLMs?

PPO/GRPO ratio $\pi_{\theta} / \pi_{\text{old}}$ (reuse rollouts for several steps), speculative decoding’s $\min(1, p/q)$, off-policy correction in async RL. Variance explodes when q under-covers $p \rightarrow$ clipping.

Q15. Which KL estimator does GRPO use and why?

$k_3 = \rho - 1 - \log \rho$: unbiased (since $\mathbb{E}_{\rho}[\rho] = 1$), always ≥ 0 , low variance. Subtlety: differentiating it through sampled tokens gives the gradient of the *other* KL direction — still a valid regularizer.

Q16. Entropy, cross-entropy, perplexity?

$H(p) = -\sum p \log p$; perplexity = e^{CE} = effective branching factor. PPL 10 $\Leftrightarrow$ loss 2.30 nats. Not comparable across tokenizers.

INTERVIEW Q&A | OPTIMIZATION AND STATISTICS

Q17. GD on a quadratic: stable step and convergence?

Each eigen-direction scales by $(1 - \eta \lambda_i)$; stable iff $\eta < 2/L$. Best fixed step gives contraction $(\kappa - 1)/(\kappa + 1)$. $\kappa = 100$: GD 0.980/step (350 steps for $10^3 \times$), heavy-ball momentum 0.818/step (35 steps) — the $\sqrt{\kappa}$ speed-up is why every optimizer has momentum.

Q18. What is the edge of stability?

Full-batch GD drives sharpness (top Hessian eigenvalue) up to $\approx 2/\eta$ and hovers there while loss keeps falling non-monotonically (Cohen et al. 2021).

Q19. Model A 71.2%, B 70.0% on 1,000 items. Better?

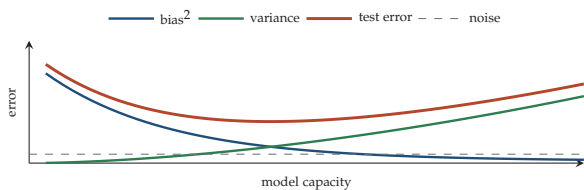
Unpaired 95% CI is ± 2.8 pts each — not established. Use a **paired** test on per-item differences (shared variance cancels), bootstrap when no formula exists, clustered SEs when items share a document, and report spread over seeds, not the best seed.

Q20. Critical batch size?

Below it, doubling batch nearly halves steps; above it, extra batch buys almost nothing (McCandlish et al. 2018). It grows as loss falls, so large runs ramp batch size.

2 ML and DL foundations interviewers still ask

[gap-fill - the screening round before the LLM round]



Classical U-curve. Over-parameterized nets show *double descent*: error peaks near the interpolation threshold, then falls again with more capacity, data or training.

INTERVIEW Q&A | LEARNING THEORY AND REGULARIZATION

Q21. Bias–variance decomposition?

$\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma^2$. High bias: underfit (bigger model, better features). High variance: overfit (more data, regularize, ensembling, early stopping).

Q22. L1 vs L2 regularization?

L2 (ridge, Gaussian prior) shrinks smoothly; L1 (lasso, Laplace prior) drives weights to exactly zero (sparsity) because its subgradient is constant near 0.

Q23. Why does dropout work?

Trains an implicit ensemble of thinned subnetworks; prevents co-adaptation; scale by $1/(1 - p)$ at train time (inverted dropout) so inference needs no change.

Q24. BatchNorm vs LayerNorm?

BN normalizes each feature over the batch (needs running stats, breaks with small/variable batches and autoregressive decoding); LN normalizes each token over features (batch-independent) $\rightarrow$ LN/RMSNorm in transformers.

Q25. Why does Adam generalize differently from SGD?

Adaptive per-parameter steps converge fast but can find sharper minima in vision; with AdamW and good schedules it is the LLM default. SGD+momentum still common in CNNs.

Q26. Cross-validation and leakage?

k -fold for small data; split by group/time to avoid leakage (same user, future data); never fit preprocessing on the full set.

Q27. Class imbalance?

Use PR-AUC/F1 not accuracy; reweight or resample; focal loss; threshold tuning on a validation set; calibrate afterwards.

METRICS

precision / recall	$\frac{TP}{TP+FP} / \frac{TP}{TP+FN}$; $F1 = \frac{2PR}{P+R}$
ROC-AUC	$P(\text{score pos} > \text{score neg})$; threshold-free; misleading under heavy imbalance
PR-AUC	better for rare positives
calibration	ECE: $\sum_b \frac{n_b}{n} \text{acc}_b - \text{conf}_b $; fix with temperature scaling
perplexity	$\exp(\text{mean NLL})$
BLEU / ROUGE	n -gram precision (brevity penalty) / recall-oriented overlap; weak for LLM outputs
BERTScore	token embedding similarity
recall@ k , MRR, nDCG	retrieval: hit in top k ; $1/\text{rank}$ of first hit; graded, log-discounted
WER	speech: (S+D+I)/N

INTERVIEW Q&A | NEURAL NETWORK HISTORY YOU’LL BE ASKED TO CONNECT

Q28. Why did RNNs/LSTMs lose to transformers?

Sequential computation (no parallelism over time), vanishing gradients over long ranges (LSTM gates help but don’t fix), fixed-size state bottleneck. Attention gives $O(1)$ path length between any two tokens and parallel training.

Q29. What did attention originally fix (Bahdanau 2014)?

The seq2seq bottleneck: the decoder attends over all encoder states instead of one context vector.

Q30. word2vec in one line?

Skip-gram with negative sampling learns embeddings so that $\sigma(u_o^T v_c)$ is high for real context pairs and low for sampled negatives; implicitly factorizes a shifted PMI matrix.

Q31. CNN essentials?

Local receptive fields, weight sharing, translation equivariance; output size $\lfloor (n + 2p - k)/s \rfloor + 1$; params $k^2 C_{\text{in}} C_{\text{out}}$. ResNets made depth trainable with identity skips — the same idea as the residual stream.

Q32. What is the universal approximation theorem’s catch?

One hidden layer can approximate any continuous function, but may need exponential width; says nothing about learnability. Depth buys efficiency.

Q33. Generative vs discriminative?

Model $p(x, y)$ (or $p(x)$) vs $p(y|x)$. LLMs are generative models used discriminatively via prompting.

Q34. What is self-supervised learning?

Labels come from the data: next-token prediction, masked LM (BERT 15% masking), contrastive (SimCLR, CLIP), masked autoencoding (MAE).

Q35. Encoder (BERT) pretraining details?

MLM: mask 15% (80% [MASK], 10% random, 10% unchanged); NSP was later shown unnecessary (RoBERTa). Used for embeddings, classification, rerankers.

INTERVIEW Q&A | PROBABILITY REFRESHERS

Q36. Bayes' rule and why it matters for evals?

$P(A|B) = P(B|A)P(A)/P(B)$. A 95%-accurate detector at 1% base rate has precision $\approx 16\%$ — always ask for the base rate (e.g. judge pass rates, safety classifiers).

Q37. Expected value of a max over n draws?

Grows with n (best-of- n , best seed, best prompt): selection inflates reported scores — report means, hold out a confirm set.

Q38. Temperature in softmax?

$p_i \propto e^{z_i/T}$: $T \rightarrow 0$ argmax, $T \rightarrow \infty$ uniform; entropy increases with T .

INTERVIEW Q&A | CLASSICAL ML IN ONE BREATH

Logistic regression loss and gradient? $\rightarrow$ BCE; $\nabla_w = X^T(\sigma(Xw) - y)/n$; convex

Linear regression closed form? $\rightarrow w = (X^T X + \lambda I)^{-1} X^T y$

Naive Bayes assumption? $\rightarrow$ features conditionally independent given the class

SVM objective? $\rightarrow$ max margin: hinge loss $+\frac{\lambda}{2}\|w\|^2$; kernels via the dual

Decision tree split criterion? $\rightarrow$ Gini or entropy (information gain); MSE for regression

Random forest vs gradient boosting? $\rightarrow$ bagging + feature subsampling (variance $\downarrow$) vs sequential fit to residuals/gradients (bias $\downarrow$)

Why do GBDTs still win on tabular data? $\rightarrow$ handle heterogenous features, missing values, monotone splits; little tuning

k-means objective and pitfall? $\rightarrow$ within-cluster SSE; local optima (k-means++ init), assumes spherical clusters

PCA? $\rightarrow$ top eigenvectors of the covariance = top right singular vectors of centered X

k-NN complexity? $\rightarrow$ no training; $O(nd)$ per query (ANN indexes for scale)

EM algorithm? $\rightarrow$ alternate posterior over latents (E) and maximize expected log-likelihood (M); monotone in likelihood

Curse of dimensionality? $\rightarrow$ distances concentrate; volume grows exponentially; need data $\propto e^d$

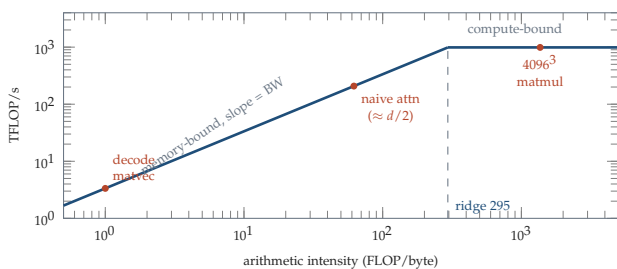
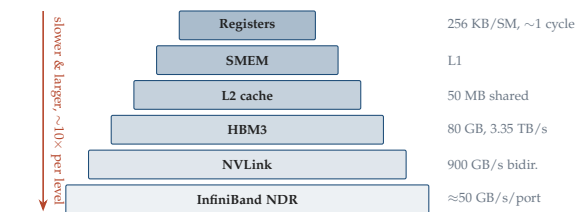
Gaussian MLE? $\rightarrow \hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n} \sum (x - \bar{x})^2$ (biased; $\frac{1}{n-1}$ unbiased)

A/B test p-value? $\rightarrow P(\text{effect at least this large} \mid \text{no true effect})$; not $P(H_0 \text{ true})$

3 Compute and hardware

[module 02 · lab 03 napkin math]

Every design decision reduces to **FLOPs, bytes and bandwidth**. Most kernels are limited by moving bytes, not doing math; every optimization below (fusion, tiling, FlashAttention, quantization, KV compression) moves fewer bytes through slow memory.



ROOFLINE

$t \geq \max \left(\frac{\text{FLOPs}_{\text{peak}}}{I}, \frac{\text{bytes}}{BW} \right) + \text{overhead}$. Intensity $I = \text{FLOPs/bytes}$; ridge = peak/BW.

Matmul: $I = \frac{mkn}{mk+kn+mn}$ (bf16). Square: $n/3$ (1365 at 4096). Decode ($m=1$): $I \approx 1$. Batched decode: $I \approx B \rightarrow$ need $B \approx 300$ to hit the H100 ridge. Elementwise ops: $I < 1 \rightarrow$ fuse them.

GPU	bf16 dense	HBM	BW	ridge
A100 80G	312 TF/s	80 GB	2.0 TB/s	150
H100 SXM	989 TF/s	80 GB	3.35 TB/s	295
B200	~ 2.2 PF/s	180 GB	~ 8 TB/s	280

NUMBER FORMATS: EXPONENT = RANGE, MANTISSA = PRECISION

fmt	s/e/m	max	ϵ	use
fp32	1/8/23	$3 \cdot 10^{38}$	$1.2 \cdot 10^{-7}$	master weights, optimizer, reductions
bf16	1/8/7	$3 \cdot 10^{38}$	$7.8 \cdot 10^{-3}$	default train/infer, no loss scaling
fp16	1/5/10	65,504	$9.8 \cdot 10^{-4}$	needs loss scaling to train
e4m3	1/4/3	448	0.125	FP8 fwd weights/acts
e5m2	1/5/2	57,344	0.25	FP8 gradients
fp4 e2m1	1/2/1	6	0.5	MXFP4 (32/blk, pow2 scale), NVFP4 (16/blk, fp8 scale)

INTERVIEW Q&A | COMPUTE AND MEMORY

Q39. Why fp32 master weights if you compute in bf16?

bf16's gap after 1.0 is 0.0078; an Adam update of 10^{-3} relative size rounds away (bf16(1)+1e-3 == 1). The fp32 copy accumulates small updates. Same reason accumulations run in fp32 (bf16(256)+1 == 256).

Q40. Why bf16 over fp16 for training?

fp16 has 5 exponent bits: overflow above 65,504, gradients underflow below $\sim 6 \cdot 10^{-8} \rightarrow$ dynamic loss scaling, still fragile on spikes. bf16 has fp32's range.

Q41. Forward and training FLOPs per token?

Forward $\approx 2N$ (one multiply-add per weight). Backward computes $\tilde{X}$ and $\tilde{W}$, each as big as the forward matmul $\rightarrow$ training $\approx 6N$; activation checkpointing adds a forward $\rightarrow 8N$. Attention adds $\approx 6LTD$ per token in training: at 128k context it exceeds $6N$ for an 8B model.

Q42. Memory to train a 7B model with AdamW?

16 B/param: bf16 weights 2 + bf16 grads 2 + fp32 master 4 + Adam m 4 + v 4 = 112 GB before activations $\rightarrow$ ZeRO/FSDP, 8-bit optimizers or LoRA.

Q43. Activation memory per layer (GPT-3 style, 16-bit)?

$sbh(34 + 5as/h)$ bytes (Korthikanti et al.). The $5as^2b$ term is the attention matrices; FlashAttention removes it. Full checkpointing stores only $2sbh$ per layer at +33% compute. Don't forget the fp32 logits: $s \cdot b \cdot V \cdot 4$ bytes.

Q44. What is MFU and what is good?

Model FLOPs ($6N$ + attention) achieved $\div$ peak. Good large runs: 35–55% (Llama 3 405B: 38–43%). HFU also counts recomputation. MFU $> 100\% \rightarrow$ missing `cuda.synchronize`, tied embedding counted twice, MoE total instead of active params, or wrong peak.

Q45. Small model runs equally fast on A100 and H100. Why?

Overhead-bound: kernels too small to fill 132 SMs; CPU launch and Python dominate. Fix with bigger batches, `torch.compile`, CUDA graphs, fusion — not a faster GPU.

Q46. Batch-1 decode speed upper bound for an 8B bf16 model?

$\text{tokens/s} \leq BW \div \text{weight bytes} = 3.35/16 \approx 200 \text{ tok/s}$, whatever the FLOPs.

COLLECTIVES (RING, n RANKS, BUFFER S)

collective	bytes/rank	where
all-reduce	$2 \frac{n-1}{n} S$	DP grads, TP partial sums
reduce-scatter	$\frac{n-1}{n} S$	ZeRO-2/3, FSDP grads
all-gather	$\frac{n-1}{n} S$	ZeRO-3/FSDP params
all-to-all	$\frac{n-1}{n} S$	MoE dispatch, Ulysses SP
send/recv	activations	pipeline parallel

All-reduce = reduce-scatter + all-gather; per-rank cost $\approx$ independent of n . 2 GB grads over 8 GPUs: 55 ms on PCIe 5 (64 GB/s), 7.8 ms on NVLink (450 GB/s/dir).

TRAIPS

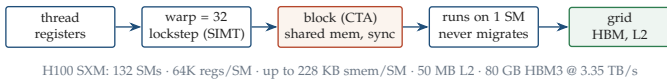
- “Make the math cheaper.” Cut bytes first; FlashAttention does *more* FLOPs and is faster.
- Quoting sparse TFLOP/s (H100 “1,979” assumes 2:4 sparsity). GB

vs GiB; Gb vs GB (IB is in gigabits).

- ▶ “Batching makes decode attention efficient.” It amortizes weights, not per-sequence KV reads.
- ▶ Timing GPU code without `torch.cuda.synchronize()` or CUDA events; not discarding warm-up iterations.

4 GPU and TPU programming

[lab 06 attention kernels · NVIDIA and Google loops · question bank §10]



INTERVIEW Q&A | CUDA EXECUTION MODEL

Q47. What is memory coalescing?

A warp's 32 loads are merged into as few 32-byte sectors as possible. 32 consecutive fp32 values = 128 B = 4 sectors (ideal). Stride-32 access touches 32 sectors for the same useful bytes: $8\times$ less effective bandwidth. Make the fastest-varying thread index walk the contiguous dimension.

Q48. Shared-memory bank conflicts?

32 banks, 4 B wide. Threads reading a column of `float t[32][32]` all hit one bank (32-way conflict, serialized). Pad to `t[32][33]` so column elements fall in different banks. Broadcast (same address) is free.

Q49. Compute occupancy for 128 registers/thread.

$65,536/128 = 512$ threads = 16 warps per SM, out of a max of 64 warps (25%). Smem per block and block count limits also apply. More occupancy hides latency but is not the goal: GEMMs trade occupancy for big register tiles (reuse).

Q50. Warp divergence?

Threads of one warp taking different branches execute both paths with masks. Make branches uniform per warp (e.g. branch on `warp_id`, not `thread_id%2`).

Q51. Arithmetic intensity of a tiled GEMM?

Per K-step a block loads $(B_M + B_N)B_K$ elements and does $2B_M B_N B_K$ FLOPs: $I = \frac{2B_M B_N}{(B_M + B_N) \cdot \text{bytes}}$. 128×128 bf16: 64 FLOP/B; 32×32 fp32: 8. Ridge on H100 ≈ 295 , so bigger tiles + L2 reuse are what make GEMMs compute-bound. One 128×128, $B_K=32$ bf16 stage = 16 KB smem; 3 pipeline stages = 48 KB.

Q52. What do tensor cores need?

Matrix fragments (e.g. 16×8×16 for bf16 mma), dimensions that are multiples of 8/16 (pad vocab to 128), and for FP8/FP4 per-tensor or per-block scales. Hopper adds `wgmma` (warpgroup = 4 warps) and TMA async copies; Blackwell adds FP4 and tensor memory.

Q53. `__syncthreads()` semantics and pitfalls?

Barrier for all threads of a *block* (not the grid). Inside a divergent branch it deadlocks or is undefined. Grid-wide sync needs a new kernel launch or cooperative groups.

Q54. Streams, events, async copies?

Kernels in different streams may overlap; events time and order them. `cp.async/TMA copy global→smem` without registers so loads for tile $k+1$ overlap compute on tile k (software pipelining, `num_stages`).

Q55. Why does a fused softmax beat three kernels?

Unfused: max, exp-sum, normalize each read/write the row from HBM. Fused keeps the row in registers/smem: one read, one write. All memory-bound ops in a chain should be fused (bias+act+residual+norm).

Q56. Nsight Compute says memory 90%, compute 20%. Next?

It's memory-bound: raise reuse (tiling, fusion), cut bytes (lower precision, avoid re-reads), check coalescing and L2 hit rate. Compute 90%/memory 20%: you are at the compute roof, so only fewer FLOPs or tensor cores help.

Q57. What are CUDA graphs for?

Record a sequence of launches once, replay with one CPU call: removes per-kernel launch overhead ($\sim 5\text{--}10\mu\text{s}$ each), crucial for small-batch decode. Requires static shapes and addresses (hence padded batch buckets).

Q58. How does NCCL all-reduce scale?

Ring: each GPU sends $2(n-1)/n \cdot S$ bytes, so time $\approx 2S/\text{BW}$ independent of n (bandwidth-optimal) plus $2(n-1)$ latency hops. Trees for small messages; NVLink/NVSwitch inside a node, IB/RoCE across.

TRITON IN EIGHT LINES (FUSED ROW SOFTMAX)

```
@triton.jit
def softmax_k(X, Y, stride, n, BLOCK: tl.constexpr):
    row = tl.program_id(0) # one program per row
    cols = tl.arange(0, BLOCK); mask = cols < n
    x = tl.load(X+row*stride+cols, mask=mask, other=-1e30)
    x = x - tl.max(x, 0); e = tl.exp(x)
    tl.store(Y + row*stride + cols, e / tl.sum(e, 0), mask=mask)
# launch: softmax_k[(rows,)](x, y, x.stride(0), n, BLOCK=1024)
```

You write a program *per tile*; the compiler maps threads, smem, coalescing and tensor-core ops.

@triton.autotune benchmarks block sizes, num_warps, num_stages per shape key.

TPU NAPKIN NUMBERS (VERIFY AGAINST CLOUD TPU DOCS)

chip	HBM	HBM BW	bf16 FLOP/s	ridge (FLOP/B)
v5e	16 GB	0.82 TB/s	197 T	≈ 240
v5p	96 GB	2.8 TB/s	459 T	≈ 164
v6e (Trillium)	32 GB	1.6 TB/s	920 T	≈ 575

$[B, D] \times [D, F]$ matmul intensity $\frac{2BDF}{2BD+2DF+2BF} \approx B$ for $B \ll D, F$: compute-bound only when the per-replica token batch exceeds the ridge (240 tokens on v5e). Sharding scales FLOPs and bandwidth together, so the threshold is per replica. TPUs: systolic MXUs (128×128 or 256×256), ICI torus links, XLA compiles everything.

PYTORCH HABIT TO JAX EQUIVALENT

<code>loss.backward()</code>	<code>jax.grad(f)(params, batch)</code> returns a new pytree
<code>eager / torch.compile</code>	<code>jax.jit</code> : traces once per shape/dtype; side effects run at trace time only
manual batch dim	<code>jax.vmap</code>
global RNG	explicit keys, <code>jax.random.split(key)</code>
DDP / FSDP	<code>Mesh + NamedSharding + jit (GSPMD)</code> ; <code>shard_map</code> for manual collectives
Triton	Pallas (TPU and GPU)
<code>nn.Module, optim</code>	Flax (NNX), Optax
in-place updates	functional: <code>x.at[i].set(v)</code> ; <code>jax.lax.scan</code> for loops

CLASSIC JAX BUGS

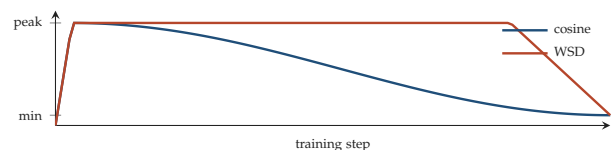
Recompiling on every new sequence length (bucket or pad shapes); Python if on traced values (use `jnp.where / lax.cond`); reusing a PRNG key; printing inside jit (use `jax.debug.print`); forgetting `block_until_ready()` when timing (async dispatch).

5 Deep learning that matters at scale

[module 03 · labs 01, 02]

DEFAULTS YOU MUST BE ABLE TO DERIVE

Kaiming (ReLU)	$\text{Var}(W) = 2/n_{\text{in}}$; LeCun (linear/tanh) $1/n_{\text{in}}$; Xavier $2/(n_{\text{in}} + n_{\text{out}})$
Residual init	scale out-projections by $1/\sqrt{2L}$ (GPT-2: $0.02/\sqrt{24} = 0.0041$)
AdamW (LLM)	$\beta = (0.9, 0.95)$, wd 0.1 (not norms/biases), clip 1.0, $\epsilon = 10^{-8}$
Adam bias corr.	$\hat{m}_t = m_t / (1 - \beta_1^t)$; first step = $\eta \text{sign}(g)$
Warmup	1–2% of steps (or a few thousand)
Cosine	decay to $\sim 10\%$ of peak; needs total steps up front
WSD	warmup–stable–decay over last 10–20%
Healthy update/weight	$\sim 10^{-3}$ per step
Init loss	$\ln V$
Optimizer state	SGD 0, SGD+mom 4, AdamW 8, Muon/Lion 4, 8-bit Adam ~ 2 B/param



INTERVIEW Q&A | INITIALIZATION AND NORMALIZATION

Q59. Derive Kaiming init.

$\text{Var}(y_i) = n_{\text{in}} \text{Var}(W) \mathbb{E}[x^2]$; with $x = \text{ReLU}(z)$ and symmetric z , $\mathbb{E}[x^2] = \text{Var}(z)/2$. Stability needs $n_{\text{in}} \text{Var}(W)/2 = 1$. 30 ReLU layers of width 256: $N(0, 1)$ init explodes to 10^{31} , $N(0, 0.01^2)$ vanishes to 10^{-29} , Kaiming stays $O(1)$.

Q60. Why scale residual branches by $1/\sqrt{2L}$?

The stream is a sum of $2L$ branch outputs (attn + MLP per layer); variance grows as $2Lv$. Scaling each branch std by $1/\sqrt{2L}$ makes total added variance depth-independent.

Q61. Pre-norm vs post-norm?

Pre-norm ($x + f(\text{norm}(x))$) keeps an identity gradient path, trains deep stacks without delicate warmup; needs a final norm before the head and the stream norm grows with depth. Post-norm (2017) can be slightly better when it trains but is fragile and needs long warmup.

Q62. LayerNorm vs RMSNorm?

RMSNorm drops centering and bias: $x/\sqrt{\text{mean}(x^2) + \epsilon} \cdot g$. Cheaper, equally good; Llama, Qwen, Mistral use it.

Q63. What is μP and why do labs use it?

Width-dependent init and per-layer LR (hidden matrices: Adam LR $\propto 1/\text{fan_in}$; output logits $\times 1/\text{width}$; attention $1/d_h$ instead of $1/\sqrt{d_h}$) so hyperparameters tuned on a narrow proxy transfer to the wide model.

Q64. How can weight decay act as an LR increase?

Weights feeding a norm are scale-invariant; gradient $\propto 1/\|W\|$, so the angular step is $\approx \eta/\|W\|^2$. Decay shrinks $\|W\| \rightarrow$ larger effective step; training settles at an equilibrium norm.

INTERVIEW Q&A | OPTIMIZERS AND SCHEDULES**Q65. Show the first Adam step has magnitude η .**

$m_1 = (1 - \beta_1)g$, $v_1 = (1 - \beta_2)g^2$; corrected $\hat{m}_1 = g$, $\hat{v}_1 = g^2 \rightarrow$ update $\eta g/|g| = \eta \text{sign}(g)$, independent of gradient scale.

Q66. Does bias correction make early steps bigger or smaller?

Depends on betas. Uncorrected with (0.9, 0.999): step 1 is $3.16 \times$ too large (correction shrinks it). With (0.9, 0.95): $0.45 \times$ (correction enlarges it).

Q67. Why $\beta_2 = 0.95$ for LLMs?

v averages $\approx 1/(1 - \beta_2)$ steps: 20 vs 1,000. After a $10 \times$ gradient spike in steady state: $\beta_2 = 0.999$ gives update 1.81 (amplified); $\beta_2 = 0.95$ gives 0.78 (damped).

Q68. Adam + L2 vs AdamW?

L2 adds $\lambda \theta$ to the gradient, then divides by $\sqrt{\hat{v}}$ — params with large gradient history get barely regularized. AdamW decouples: $\theta \leftarrow \theta(1 - \eta\lambda) - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$.

Q69. What is Muon?

Momentum on each 2-D hidden matrix, orthogonalized by Newton-Schulz ($USV^T \rightarrow UV^T$) so every singular direction moves equally; embeddings, head, norms stay on AdamW. 4 B/param state.

Q70. Why warmup?

Early curvature is high and Adam's $\hat{v}$ is a noisy estimate (RAdam's analysis); full steps can throw the model somewhere unrecoverable.

Q71. When WSD over cosine?

Unknown final step count, want to continue later, or need several checkpoints at different token budgets from one run (scaling-law sweeps). Branched cooldowns match cosine.

Q72. How do you scale LR with batch?

SGD: linearly (up to a point). Adam: $\approx \sqrt{k}$ for a $k \times$ batch as a start, re-tune near the critical batch.

INTERVIEW Q&A | MIXED PRECISION AND AUTODIFF**Q73. Describe a bf16 mixed-precision step.**

Matmuls in bf16 on tensor cores; softmax, norms, losses, reductions in fp32; fp32 master weights and optimizer states. No scaler needed.

Q74. fp16 loss scaling procedure?

Multiply loss by S (2^{16}) before backward; **unscale before clipping**; on inf/NaN skip the step and halve S ; double after $\sim 2,000$ good steps.

Q75. What does `loss.backward()` actually do?

Walks the define-by-run DAG in reverse topological order applying each `grad_fn`'s VJP, *summing* where paths merge, *accumulating* into leaf `.grad` (hence `zero_grad`), freeing saved tensors (second backward needs `retain_graph`). In-place edits to saved tensors bump a version counter and error.

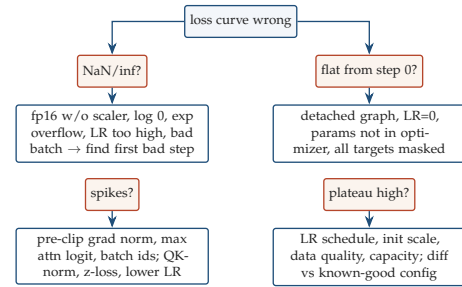
Q76. Gradient accumulation normalization?

Divide by total valid tokens across all micro-batches, not by micro-batch count, or short sequences get over-weighted.

Q77. Activation checkpointing trade-off?

Store every k -th layer input, recompute the rest: $k \approx \sqrt{L}$ gives

$O(\sqrt{L})$ memory for one extra forward ($\approx +33\%$ compute). Selective recompute only redoes the cheap-FLOP, big-memory attention parts.

Debugging a run from its curves**SYMPTOM $\rightarrow$ CAUSE**

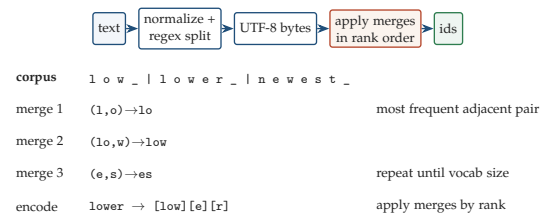
init loss $\gg \ln V$	LM head too large, no final norm, μP multiplier inverted
init loss $\ll \ln V$	label leakage: targets not shifted, causal mask missing
stuck at exactly $\ln V$	output independent of input: detached graph, wrong targets
falls then slowly rises	LR too high late, no decay; train/eval mismatch
spikes at same step across seeds	bad data shard at a fixed position
random growing spikes	attention/output-logit growth $\rightarrow$ QK-norm, z-loss
drop at each epoch	memorizing repeated data
val < train	dropout only in train, leaked/easier eval
tokens/s decays	loader starvation, leaks, fragmentation

TRAPS

- Tuning hyperparameters before you can **overfit one batch**. Most bugs are data bugs: decode a batch back to text.
- "Kaiming is $1/n_{in}$." That is LeCun; ReLU needs $2/n_{in}$.
- "Checkpointing halves memory for free." +33% compute, activations only.
- Clipping before unscaling in fp16: clipping fires every step and the real update is $65,536 \times$ too small.
- "Adam's ϵ is irrelevant." It floors the denominator when gradients are tiny (bf16, big models).

6 Tokenization

[module 04 §1, 05 §2 · lab 04 tokenizer]

**FAMILIES**

BPE (byte-level)	greedy merges of most frequent pair; 256 base bytes $\rightarrow$ no <code><unk></code> , ever. GPT-2/4, Llama 3 (tiktoken, 128k)
WordPiece	merge pair maximizing $\frac{c(ab)}{c(a)c(b)}$ (likelihood gain); ## continuation; BERT
Unigram LM	start big, prune tokens whose removal hurts likelihood least; Viterbi/sampled segmentation; SentencePiece (T5, Gemma)
SentencePiece	treats input as raw stream incl. spaces (<code>_</code>); BPE or Unigram inside
Byte/char-level	no vocab issues, $\sim 4 \times$ longer sequences (ByT5; BLT uses dynamic byte patches)

INTERVIEW Q&A | TOKENIZERS**Q78. Why byte-level BPE?**

Every string is representable (256 byte fallback), so no unknown

token; merges learned from data compress common sequences. The cost: rare scripts fragment into many bytes.

Q79. What does the pre-tokenization regex do?

Splits text (letters, digits in groups of ≤ 3 , punctuation, whitespace) so merges never cross those boundaries — e.g. “dog.” and “dog!” share “dog”. It is frozen into the model.

Q80. Vocab size trade-off?

Bigger $V \rightarrow$ fewer tokens per text (cheaper long context, faster decode per character) but a bigger embedding + LM head ($2dV$ FLOPs per token; 6.5% of Llama-3-8B’s forward) and rarer, under-trained tokens. 32k (Llama 2) $\rightarrow$ 128k (Llama 3) $\rightarrow$ 256k (Gemma).

Q81. What is fertility and why does it matter?

Tokens per word. High-fertility languages (e.g. Telugu under c100k, split at every vowel sign) pay more per word in cost and context, and get worse quality. Lab 04 measures this.

Q82. Why are LLMs bad at counting letters or arithmetic?

They see tokens, not characters; digit grouping is arbitrary (“1234” may be one or two tokens). Fixes: single-digit tokenization, right-to-left grouping, tool use.

Q83. What are glitch tokens?

Tokens in the vocab that almost never appear in training data (e.g. “SolidGoldMagikarp”) $\rightarrow$ untrained embeddings $\rightarrow$ bizarre behavior. Caused by training the tokenizer on different data than the model.

Q84. Special tokens and chat templates?

BOS/EOS, `<|im_start|>`-style role markers, tool-call tokens. The template must match training exactly; a missing EOS or a role mismatch silently degrades SFT and inference. Never let user text be parsed into special tokens (injection).

Q85. Can you change a trained model’s tokenizer?

Not cheaply: embeddings are tied to ids. Adding tokens needs new embedding rows (init as mean of sub-token embeddings) and fine-tuning; swapping tokenizers needs transplant methods + substantial training.

Q86. How do you compare losses across tokenizers?

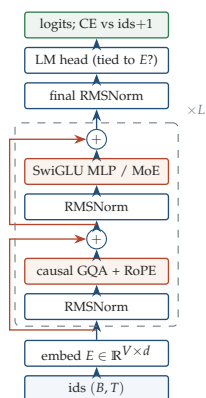
Bits per byte: $\text{BPB} = \frac{\text{total nats}}{\ln 2 \cdot \# \text{bytes}}$. Per-token loss rewards coarse tokenizers.

TRAPS

- Tokenizer and model trained on different normalization (NFC/NFKC) or whitespace handling $\rightarrow$ silent quality loss.
- Trailing-space prompts: “Answer:” produces a token the model rarely saw before the answer.
- Counting the input embedding as FLOPs (it is a lookup), but forgetting the LM head (a matmul).

7 The Transformer, piece by piece

[module 04 · labs 05 GPT, 06 kernels, 07 KV cache, 18 MoE]



Tokens in, logits out. Byte-level BPE $\rightarrow$ ids $\rightarrow$ embedding $\rightarrow L$ pre-norm blocks that read from and write to the **residual stream** $(B, T, d) \rightarrow$ final norm $\rightarrow$ LM head $\rightarrow V$ logits per position.

One forward pass gives a loss at *every* position at once: decoder-only pretraining is data-efficient per FLOP.

- Init loss = $\ln V$.
- LM head = $2dV$ FLOPs/token (6.5% of Llama-3-8B).
- Tokenizer is frozen forever.
- MLP holds $\approx 2/3$ of non-embedding params.
- Red arrows: identity residual paths (why pre-norm trains deep).

ATTENTION

$\text{Attn}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_h}} + M)V$, $M = 0$ on/below diagonal, $-\infty$ above. Heads concatenated, projected by W_O .

$\text{Var}(q \cdot k) = d_h$ for unit-variance entries $\rightarrow$ std 11.3 at $d_h=128 \rightarrow$ near one-hot softmax (mean max-prob 0.79 over 1,024 keys), Jacobian ≈ 0 . Scaling restores unit variance (max-prob 0.017).

FLOPs/layer fwd: projections $2T(2d^2 + 2dH_{kv}d_h)$; scores $2T^2d$; PV $2T^2d$.

QK circuit $W_Q W_K^T$ (where to look), OV circuit $W_V W_O$ (what to move);

each rank $\leq d_h$.



KV CACHE: $2 \cdot L \cdot H_{kv} \cdot d_h$ BYTES PER TOKEN

variant (Llama-3-8B shape: $L=32, d_h=128$) per token 8k ctx $\times$ 32 seqs

MHA, $H_{kv}=32$	512 KiB	128 GiB
GQA, $H_{kv}=8$ (Llama 3)	128 KiB	32 GiB
MQA, $H_{kv}=1$	16 KiB	4 GiB
MLA (DeepSeek-V2, 60 layers, 576/layer)	67.5 KiB	1.8% of MHA

GQA/MQA/MLA cut KV memory and decode bandwidth, not attention FLOPs.

INTERVIEW Q&A | ATTENTION

Q87. Why divide by $\sqrt{d_h}$ and not d_h ?

Dot product of two d_h -dim unit-variance vectors has variance d_h ; dividing by $\sqrt{d_h}$ gives unit variance *at init*. It does not stop logits growing during training — QK-norm does. (μP uses $1/d_h$ for width transfer.)

Q88. Why multiple heads?

H heads of size d/H cost the same FLOPs as one head of size d but give H independent attention patterns (low-rank bilinear forms).

Q89. Why does GQA speed decode but not prefill?

Decode reads the whole KV cache for one query per head ($I \approx 1$, bandwidth-bound); sharing each KV head across g query heads cuts bytes by g ($I \approx g$). Prefill is already compute-bound and FLOPs are unchanged.

Q90. Why GQA rather than MQA?

MQA (1 KV head) hurts quality and stability, and must be replicated across TP ranks; GQA-8 keeps near-MHA quality with $4\times$ less cache and shards cleanly up to $TP = H_{kv}$.

Q91. Explain MLA.

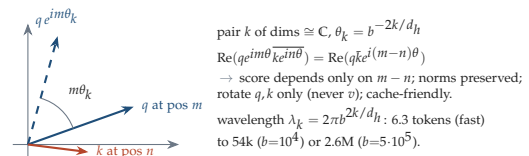
Cache a low-rank latent $c_t = W^{DKV} h_t$ ($d_c = 512$); $k = W^{UK} c$, $v = W^{UV} c$. Since $q^T W^{UK} c = (W^{UKT} q)^T c$, absorb W^{UK} into the query and W^{UV} into $W_O \rightarrow$ attend directly over c . RoPE would sit between W^{UK} and c and break absorption $\rightarrow$ a small decoupled RoPE key (64-d) shared across heads.

Q92. Sliding-window attention?

Local layers cache only W tokens (Mistral; Gemma interleaves local/global). Receptive field grows $L \cdot W$ through depth, but not reliably.

Q93. Causal mask without explicit positions — does order leak?

Some (a token can count predecessors), which is why NoPE works at small scale; but models need explicit positions to be good.



CONTEXT EXTENSION: WHICH ROPE PAIRS TO CHANGE

PI	positions m/s : all θ_k/s ; crowds fast pairs $\rightarrow$ needs fine-tuning ($\sim 1k$ steps to 32k)
NTK-aware	raise base $b' = b s^{d_h/(d_h-2)}$: fast pairs untouched, slowest scaled by s
YaRN	NTK-by-parts ($\alpha=1, \beta=32$) + temperature $\sqrt{1/t} = 0.1 \ln s + 1$ ($s=16$: logits $\times 1.63$)
ABF	large base before long training (Code Llama 10^6 ; Llama 3 $5 \cdot 10^5$)
ALiBi / NoPE	linear distance penalty (recency bias) / no positions

INTERVIEW Q&A | POSITIONS, MLP, PARAMETERS

Q94. Why does RoPE need PI/NTK/YaRN if it is “relative”?

Beyond the training length the slow pairs ($\lambda_k > L$) see rotation angles never seen in training. Fast pairs have seen all angles. Each

method is a policy for which pairs to rescale; long-context *data* is still needed to learn to use the range.

Q95. Two RoPE conventions?

Interleaved pairs $(2i, 2i+1)$ (paper, Meta) vs halves $(i, i+d_h/2)$ (HF). Loading across them without permuting W_Q, W_K runs fine and outputs garbage.

Q96. Size SwiGLU's hidden layer.

$W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x)$ has $3df$ params; match $8d^2 \rightarrow f = 8d/3$, rounded (Llama 2: 11,008; Llama 3: $\times 1.3 \rightarrow 14,336$). Gated beats ReLU/GELU at equal params; no crisp theory — say so.

Q97. Count Llama-3-8B's parameters.

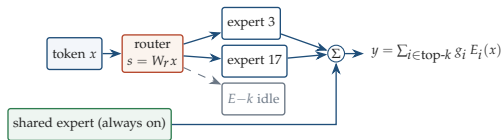
$d=4096, L=32, H=32, H_{kv}=8, f=14336, V=128256$. Attn $2d^2 + 2d \cdot 1024 = 41.9\text{M}$; MLP $3df = 176.2\text{M}$; layer $218.1\text{M} \times 32 = 6.98\text{B}$; embed+head $2Vd = 1.05\text{B} \rightarrow 8.03\text{B}$. Classic MHA + $4d$ GELU: $12d^2/\text{layer}$.

Q98. What do MLPs do, mechanistically?

Key-value memories: rows of $W_{\text{up}}/W_{\text{gate}}$ detect patterns, columns of W_{down} write associated outputs; much factual recall lives here.

Q99. Encoder-only vs decoder-only vs encoder-decoder?

BERT: bidirectional, MLM, great embeddings/classifiers, no generation. GPT: causal LM, loss at every position, scales best, does everything via prompting. T5/Whisper: encoder reads input bidirectionally, decoder cross-attends; good for translation/speech/seq2seq.



INTERVIEW Q&A | MIXTURE OF EXPERTS

Q100. Total vs active parameters?

Mixtral $8 \times 7\text{B}$: top-2 of 8, 46.7B total / 12.9B active. DeepSeek-V3: 1 shared + 256 routed, top-8, 671B / 37B. FLOPs scale with active, memory with total.

Q101. Write the Switch load-balancing loss.

$\mathcal{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$; f_i = dispatch fraction (non-differentiable), P_i = mean router prob (differentiable). Balanced value = $\alpha (10^{-2})$. Gradient $\alpha E f_i$ pushes probability off overloaded experts.

Q102. Capacity factor?

Each expert takes $\lceil \text{CF} \cdot kT/E \rceil$ tokens; overflow tokens are dropped (skip the MLP via the residual). $T=4096, E=64, k=2, \text{CF}=1.25 \rightarrow 160$ slots.

Q103. Router z-loss and aux-loss-free balancing?

z-loss $\frac{1}{T} \sum (\log \sum e^{g_i})^2 (10^{-3})$ keeps router logits small for bf16 stability. DeepSeek-V3: per-expert bias added for top- k selection only, nudged by $\pm \gamma$ by load — a controller instead of a gradient fighting the LM loss.

Q104. Why fine-grained + shared experts?

More, smaller experts $\rightarrow$ combinatorial flexibility (top-8 of 64: 4.4×10^9 combos vs 120 for top-2 of 16) at equal FLOPs; shared experts hold common knowledge so routed ones specialize.

Q105. Why are MoEs cheap to train but hard to serve?

All experts resident (671B $\approx$ 671 GB FP8); at small batch each expert sees few tokens $\rightarrow$ memory-bound on expert weights; two all-to-all per layer ($\approx 2kd$ elements/token) add latency.

STABILITY TRICKS IN CONFIGS

QK-norm	RMSNorm on q, k per head: stops attention-logit growth (OLMo 2, Gemma 3)
z-loss $10^{-4} \log^2 Z$	output-logit divergence (PaLM)
soft-capping $c \tanh(z/c)$	Gemma 2: attn 50, final 30
no biases; no wd on norms	fewer drifting params; gains don't shrink

INTERVIEW Q&A | BEYOND VANILLA ATTENTION

Q106. Why is linear attention an RNN?

Replace $\exp(q \cdot k)$ by $\phi(q)^\top \phi(k)$: $S_t = S_{t-1} + \phi(k_t) v_t^\top$, $y_t \propto \phi(q_t)^\top S_t$ — fixed-size state, $O(T)$ time, $O(1)$ memory per token.

Q107. What makes Mamba "selective"?

SSM $h_t = \bar{A} h_{t-1} + \bar{B} x_t$ with Δ, B, C functions of x_t and $\bar{A} = \exp(\Delta A)$: large Δ resets to focus on now, small Δ remembers. Parallel scan for training.

Q108. Why are pure SSMs worse at recall?

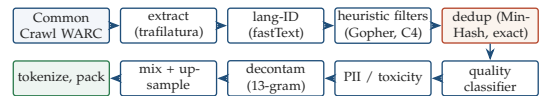
Fixed state: Mamba layer ($d=4096$, expand 2, state 16) holds 131k numbers = the KV of just 64 tokens of a GQA layer. Exact copying/needle lookup suffers $\rightarrow$ hybrids keep a few attention layers (Jamba, Nemotron-H).

TRAPS

- ▶ " $\sqrt{d_h}$ normalizes the output." It normalizes pre-softmax logits at init.
- ▶ "Apply RoPE to V ." Nothing is dotted with v .
- ▶ "SwiGLU hidden = $4d$." $\approx 8d/3$, rounded.
- ▶ "MoE is cheaper." Per training FLOP at fixed quality; not in memory, comms or small-batch serving.
- ▶ Counting tied embeddings twice; forgetting the LM head's $2dV$ FLOPs.

8 Pretraining: data, scaling, parallelism

[module 05 · labs 08 scaling laws, 09 parallelism]



FineWeb funnel: 96 snapshots $\rightarrow$ 36T tokens after base filters $\rightarrow$ 20T after per-snapshot MinHash $\rightarrow$ 15T released; FineWeb-Edu (classifier-kept) $\approx$ 1.3T and trains much better on knowledge benchmarks. Llama 3 mix $\approx$ 50% general, 25% math/reasoning, 17% code, 8% multilingual.

INTERVIEW Q&A | DATA

Q109. How does MinHash near-dedup work?

Shingle docs (word 5-grams); $\Pr[\min h(A) = \min h(B)] = J(A, B)$ (Jaccard). $k = b \cdot r$ hashes in b bands of r ; candidate if any band fully matches: $P = 1 - (1 - s^r)^b$, threshold $\approx (1/b)^{1/r}$. FineWeb: $112 = 14 \times 8 \rightarrow \sim 0.75$ similarity. Cluster candidates, keep one per cluster.

Q110. Is global dedup always better?

No (FineWeb): global dedup across 96 crawls removed up to 90% of old snapshots and barely helped; survivors were disproportionately one-off spam, while good pages recur. Dedup per snapshot.

Q111. How much can you repeat data?

Up to ~ 4 epochs is nearly as good as fresh at fixed compute (Muenighoff 2023); beyond that returns decay to zero. Always tabulate epochs per source: 50B math tokens at 10% of a 2T run = 4 epochs.

Q112. How do you decontaminate, and what slips through?

Drop/flag docs sharing n -grams (GPT-3: 13-grams) with test sets; report clean vs contaminated subsets. Paraphrases, translations and forum solutions slip through $\rightarrow$ prefer post-cutoff and private evals.

Q113. What is mid-training / annealing?

During the final LR decay, upsample highest-quality data (math, code, instruction-like, long docs). Per token it moves benchmarks most.

Q114. Packing and document masking?

Concatenate docs with EOS into fixed-length sequences (no padding waste); mask attention across document boundaries (matters most at long context).

Q115. Role and risk of synthetic data?

Rephrased web, textbook-style text, reasoning traces are central now; they inherit the generator's blind spots and narrow diversity $\rightarrow$ keep a real-data baseline, measure diversity.

SCALING LAWS

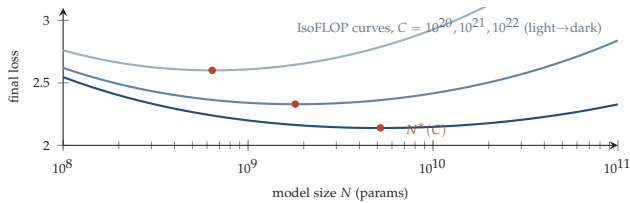
$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (\text{Chinchilla})$$

$E=1.69, A=406.4, B=410.7, \alpha=0.34, \beta=0.28$.

With $C = 6ND$: $N^* \propto C^{\beta/(\alpha+\beta)}$, $D^* \propto C^{\alpha/(\alpha+\beta)}$. Approaches 1–2: $a \approx 0.5 \rightarrow \approx 20 \text{ tokens/param}$ (Chinchilla = 70B on 1.4T).

C (approach-3 fit)	N^*	D^*	tok/param	loss
10^{20}	0.64B	26B	40	2.60
10^{22}	5.2B	323B	63	2.14
$5.76 \cdot 10^{23}$	32B	3.0T	93	1.93

Inference-aware: minimize $6ND + 2NI$. Llama 3 8B saw 15T tokens ($\approx 1,900/\text{param}$) — correct for a model served billions of times.



INTERVIEW Q&A | SCALING

Q116. Derive compute-optimal N^* .

Substitute $D = C/6N$, set $dL/dN = 0$: $\alpha AN^{-\alpha-1} = \beta B(C/6)^{-\beta} N^{\beta-1} \rightarrow N^* = G(C/6)^{\beta/(\alpha+\beta)}$, $G = (\alpha A/\beta B)^{1/(\alpha+\beta)}$.

Q117. How do you run an IsoFLOP sweep?

5–9 budgets over ≥ 1.5 decades; 6–10 sizes each trained for $D = C/6N$ with LR schedule sized to its own length; fit a parabola of loss vs $\log N$, vertex = $N^*(C)$; fit $\log N^*$ vs $\log C$. Bootstrap CIs on the exponent.

Q118. Why did Kaplan (2020) and Chinchilla (2022) disagree?

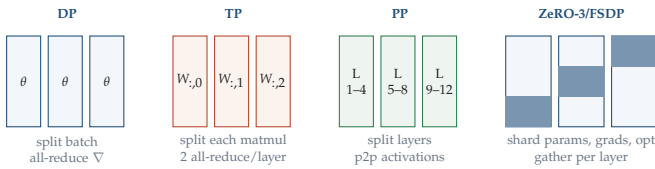
Kaplan's smaller-scale accounting (excluded embedding params/last-layer FLOPs), warmup and per-scale tuning differences (Porian et al.); not mainly the LR schedule. Kaplan said "scale N faster"; Chinchilla said scale both equally.

Q119. Is "20 tokens/param" a law?

No: it is training-compute-optimal under one protocol. Inference-heavy models should train far longer; small cost (6.72B on 2T: loss 2.023 vs 2.016 for optimal 13.2B, at half the serving cost).

Q120. Do downstream tasks scale smoothly?

Loss does; tasks are noisier, and discontinuous metrics (exact match) make smooth loss gains look like "emergence". Predict loss, then map loss to tasks.



PARALLELISM CHEAT TABLE

strategy	shards	communication
DP	batch	all-reduce grads, overlapped with backward
ZeRO-1/2/3	opt / +grads / +params	reduce-scatter grads; ZeRO-3 adds param all-gathers (1.5× DP)
TP	matmuls (col, then row)	2 all-reduces per layer per pass; NVLink only
SP	norm/dropout acts by seq	paired with TP
PP	layers into p stages	p2p; bubble $\frac{p-1}{m+p-1}$ (16% at $p=4, m=16$)
CP	sequence inside attention	KV blocks passed around a ring
EP	MoE experts	all-to-all dispatch + combine

ZeRO per GPU, 7.5B on 64 GPUs: 120 → 31.4 → 16.6 → 1.9 GB.
Global batch = DP × micro-batch × grad-accum.

INTERVIEW Q&A | DISTRIBUTED TRAINING

Q121. Order of decisions for a parallel layout?

(1) Does one replica fit (16 B/param + activations)? If not, shard state in-node (FSDP) or $TP \leq \text{GPUs/node}$, then PP across nodes. (2) Does one sequence fit? SP, then CP. (3) Scale throughput with DP/HSDP. (4) Check every collective overlaps or costs <10% of step. Llama 3 405B: $TP8 \times PP16 \times CP \times DP$, 38–43% MFU on 16k H100s.

Q122. Why never TP across nodes?

Its all-reduces sit on the critical path of every layer; NVLink ~450 GB/s/dir vs IB ~50 GB/s (9×). Example: 17 ms in-node vs 150 ms over IB against 52 ms of compute.

Q123. Explain Megatron TP for an MLP.

Split W_1 by columns (each GPU computes part of the hidden, elementwise GELU stays local), W_2 by rows (partial sums) → one all-reduce forward, one backward per block. Attention splits by heads.

Q124. Does ZeRO-2 cost more communication than DP?

No: all-reduce = reduce-scatter + all-gather; ZeRO-2 does RS of grads + AG of updated params — same bytes. ZeRO-3/FSDP adds a param all-gather in backward: 1.5×, per micro-batch.

Q125. How do you shrink the pipeline bubble?

More micro-batches m (bubble $(p-1)/(m+p-1)$), interleaved/virtual stages, 1F1B schedules, zero-bubble schedules (DualPipe in DeepSeek-V3).

Q126. Plan a 7B run: 2T tokens, 256 H100s.

6.72B params ($d4096$, L32, GQA-8, V128k). ≈ 40.4 GFLOP/token → $C = 8.1 \cdot 10^{22}$. At 40% MFU: 9.2 days, 57k GPU-h. Batch 4.2M tokens, 477k steps, AdamW 3e-4, WSD. HSDP: FSDP in-node × 32 replicas; ~ 40 GB/GPU. Checkpoint every ~ 2 h; spend 5–10% on proxies first.

INTERVIEW Q&A | STABILITY, FAILURES, LONG CONTEXT

Q127. A loss spike on day 6. What do you do?

Check grad norm and max-logit history one or two steps before, which layers; rewind ~ 100 steps, skip the offending 200–500 batches (PaLM did ~ 20 times), lower LR if it repeats. Prevent: QK-norm, z-loss, smaller ϵ .

Q128. How often to checkpoint?

Young: $\tau \approx \sqrt{2\delta M}$. $\delta=1$ min, MTBF 8.4 days → ~ 155 min. Llama 3 405B: 419 unexpected interruptions in 54 days (78% hardware).

Q129. How do you get a 128k-context model?

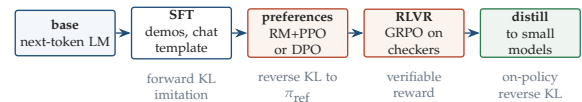
Pretrain at 4–8k with a large RoPE base; extend in stages (Llama 3: 6 stages, ~ 800 B tokens) with long docs while keeping short data; CP/ring attention for memory; evaluate with RULER-style tasks and report *effective* context, not needle-only.

TRAPS

- ▶ Using Chinchilla's published A, B, α, β and expecting 20 tokens/-param (that fit gives 40–100).
- ▶ "FSDP costs the same as DP." Not ZeRO-3: 1.5×, per micro-batch.
- ▶ Changing GPU count and silently changing global batch (= changing the optimization).
- ▶ Treating hardware failure as rare: at 16k GPUs it is daily; restart is part of the design.

9 Post-training: SFT, PEFT, RLHF, DPO, GRPO

[module 06 - labs 10 LoRA, 11 DPO, 12 GRPO]



INTERVIEW Q&A | SFT

Q130. What is the SFT loss, precisely?

$-\sum_{t \in \text{response}} \log \pi_{\theta}(y_t | x, y_{<t})$; prompt tokens fed in but masked (label =100). It is forward KL to the data → mode-covering: imitates everything, including mistakes.

Q131. Most common silent SFT bugs?

Template mismatch train vs serve; end-of-turn token masked out (pad = eos, collator masks pads) → model never stops; BOS added twice; loss on prompt tokens. Loss goes down, model gets worse. Decode a batch with masks shown.

Q132. Packing correctly?

Block-diagonal causal mask + position ids restarting at 0 per example (varlen FlashAttention takes `cu_seqLens`).

Q133. SFT data recipe?

Human demos, stronger-model synthetic data, rejection sampling (STaR); decontaminate; mix > size; 1–3 epochs; keep some general chat to limit forgetting. Read 50 random examples first.

Q134. Why does SFT on stronger-model outputs cause hallucination?

It teaches the student to state facts the teacher knew but the student does not: it imitates confidence without the knowledge.



INTERVIEW Q&A | PARAMETER-EFFICIENT FINE-TUNING

Q135. Why init $B = 0$, A random?

$BA = 0$ so training starts exactly at the base model. Both random

→ perturbed start; both zero → no gradient for either ($\nabla A \propto B^\top$, $\nabla B \propto A^\top$).

Q136. Why the α/r scale?

Keeps update magnitude roughly independent of r at fixed α . rsLoRA uses $\alpha/\sqrt{r}$ so higher ranks actually learn more.

Q137. Where to put LoRA, and what LR?

All linear layers incl. MLP (attention-only underperforms). LR $\approx 10 \times$ full fine-tuning. Qwen2.5-0.5B, $r=16$ all 7 linears: 8.8M params (1.8%).

Q138. What can't LoRA do?

On large continued-pretraining-style data (code, math at scale) it learns less — and forgets less. For RL it matches full FT even at tiny rank (an episode carries few bits).

Q139. Explain QLoRA.

Frozen base in NF4 (16 levels at normal quantiles, block 64 absmax), dequantized to bf16 per matmul; bf16 LoRA on top; double quantization of scales; paged optimizers. ≈ 4.5 bits/param → 7–8B on an 8 GB card.

Q140. Serving 1,000 customer fine-tunes?

Keep one base; batch requests with different adapters in one forward (S-LoRA, Punica). Alternatives: prefix/prompt tuning, IA³, DoRA (magnitude/direction split).

RLHF OBJECTIVE AND ITS CLOSED FORM

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta \text{KL}(\pi \| \pi_{\text{ref}}) \Rightarrow \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{r(x,y)/\beta}$$

RM loss (Bradley–Terry): $-\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$; only differences identified.

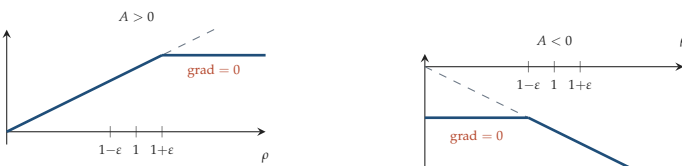
Per-token PPO reward: $r_t = -\beta \log \frac{\pi_{\theta}(y_t)}{\pi_{\text{ref}}(y_t)} + \mathbb{1}[t=T] r_{\phi}(x, y)$.

PPO-clip: $\mathbb{E}[\min(\rho A, \text{clip}(\rho, 1 \pm \epsilon) A)]$, $\epsilon=0.2$.

GAE: $\delta_t = r_t + \gamma V_{t+1} - V_t$, $\hat{A}_t = \sum_i (\gamma \lambda)^i \delta_{t+i}$.

DPO: $-\log \sigma(\beta \log \frac{\pi_{\theta}(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_{\theta}(y_l)}{\pi_{\text{ref}}(y_l)})$, $\beta \in [0.01, 0.5]$.

GRPO: $\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ per group of G ; PPO-clip; $-\beta k_3$.



INTERVIEW Q&A | RLHF AND PPO

Q141. What does the PPO clip actually do?

Removes the incentive to move further in the improving direction once ρ leaves $[1 - \epsilon, 1 + \epsilon]$ (flat regions above). It never clips harmful moves: with $A < 0$ and large ρ , ρA is smaller and its gradient pulls back. It does not bound the change — monitor clip fraction and KL.

Q142. Why the KL penalty?

Keeps the policy where the RM is still accurate, preserves fluency/diversity. Gao et al.: proxy reward keeps rising with KL while gold reward peaks then falls — Goodhart with a curve.

Q143. How many models does PPO-RLHF hold?

Four: policy (train), value (train), reference (frozen), reward (frozen) — the cost DPO and GRPO remove.

Q144. GAE λ ?

$\lambda = 0$: one-step TD (low variance, biased by V); $\lambda = 1$: Monte Carlo (unbiased, high variance). LLMs often use $\gamma = 1$.

Q145. RM accuracy ceiling?

Inter-annotator agreement (often 65–75%), not 100%. RM = SFT model with a scalar head at the last token.

Q146. RLHF failure modes?

Reward hacking, length exploitation, sycophancy, mode collapse (track entropy, distinct- n).

INTERVIEW Q&A | DPO

Q147. Derive DPO in five steps.

(1) Optimum $\pi^* = \pi_{\text{ref}} e^{r/\beta} / Z$ (rewrite objective as $-\beta \text{KL}(\pi \| \pi^*) + \beta \log Z$). (2) Invert: $r = \beta \log \frac{\pi^*}{\pi_{\text{ref}}} + \beta \log Z(x)$. (3) Plug into Bradley–Terry: $Z(x)$ cancels in $r_w - r_l$. (4) Replace π^* by π_{θ} , maximize likelihood of preferences. (5) Gradient weights each pair by $\sigma(\hat{r}_l - \hat{r}_w)$: mis-ordered pairs dominate.

Q148. “DPO has no reward model so it optimizes no reward.” True?

False: it optimizes the same KL-regularized objective, reward reparameterized through the policy (implicit reward $\beta \log \pi_{\theta} / \pi_{\text{ref}}$). It removes the explicit RM and sampling, not the objective.

Q149. DPO failure modes?

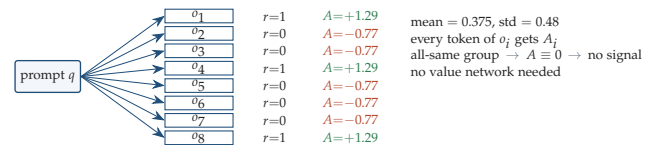
Likelihood displacement (both chosen and rejected log-probs fall; mass goes elsewhere); deterministic preferences push $\pi(y_l) \rightarrow 0$ regardless of β (IPO fixes); length exploitation (SimPO normalizes); off-policy pairs (online/iterative DPO is better).

Q150. DPO debugging checklist?

Same template/prompt for chosen and rejected; accuracy up while chosen log-prob collapses?; chosen systematically longer?; π_{ref} = your SFT model, log-probs summed over response tokens with the shift.

THE PREFERENCE FAMILY

method	ref?	idea
DPO	yes	closed-form RLHF optimum, $-\log \sigma(\beta h)$
cDPO	yes	label smoothing for noisy pairs
IPO	yes	regress margin to $1/2\tau$: certain prefs don't blow up
KTO	yes	unpaired good/bad labels, prospect theory
SimPO	no	length-normalized log-prob + margin γ
ORPO	no	SFT + odds-ratio term in one stage
Online DPO	yes	fresh pairs from current policy + judge



INTERVIEW Q&A | RLVR AND GRPO

Q151. Why GRPO instead of PPO for reasoning?

A value network for long CoT with one terminal reward is hard to learn and costs a policy-sized model. The group mean is a Monte Carlo estimate of the prompt's value.

Q152. Why must GRPO rollouts be sampled at temperature > 0?

At $T = 0$ all G samples are identical → every advantage 0 → nothing trains.

Q153. Known GRPO problems and fixes?

Length bias from per-sequence $1/|o_i|$ (long wrong answers penalized less per token) → token-level or constant normalization (DAPO, Dr. GRPO). Std normalization over-weights too-easy/too-hard prompts → drop it (Dr. GRPO). Entropy collapse → clip-higher $\epsilon_{\text{high}}=0.28$. Zero-variance groups → dynamic sampling. Truncated answers → overlong filtering. KL holds back long-CoT → $\beta=0$. Token-ratio noise (MoE) → sequence-level ratios (GSPO).

Q154. How do you design an RLVR reward?

Correctness is the reward, format a gate; normalize and compare answers symbolically; code runs hidden tests in a sandbox; unparseable or multiple answers = 0; avoid partial credit; keep prompts with pass rate strictly between 0 and 1.

Q155. Reward hacking examples?

Listing several answers for a lenient extractor; special-casing visible tests, deleting tests, exiting early with success; flattering an LLM judge; unbounded length. Defence: read samples, stricter held-out grader, track the gap. Don't optimize against the CoT — it teaches obfuscation.

Q156. Does RL create reasoning or select it?

Evidence (Yue et al.): RLVR beats base at pass@1 but base catches up at large k → mostly sharpening; longer RL (ProRL) may expand it. Know both sides.

Q157. What to log every RL step?

Mean reward, zero-variance-group fraction, length split by correct/incorrect, entropy, clip fraction, KL to reference.

INTERVIEW Q&A | TEST-TIME COMPUTE AND DISTILLATION

Q158. Ways to buy accuracy with inference compute?

CoT; self-consistency (majority of N); best-of- N with a verifier/RM; PRM-guided beam/tree search; budget forcing (s1: append “Wait”). Compute-optimal: easy/medium problems gain most; hardest need

pretraining compute.

Q159. ORM vs PRM?

Outcome RM scores the final answer; process RM scores each step (PRM800K). PRMs select better but are costly to label and hackable as RL rewards.

Q160. Forward-KL vs reverse-KL vs on-policy distillation?

Sequence-level KD = SFT on teacher samples (forward, mode-covering). Logit KD at temperature T (scale loss by T^2 , same tokenizer). On-policy: sample from the student, teacher scores each token $\rightarrow$ dense per-token reward, no exposure bias (GKD, Qwen3).

Q161. Constitutional AI / RLAIIF?

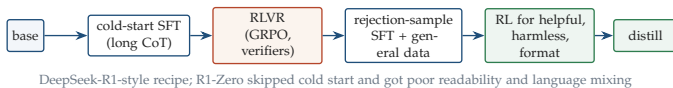
Model critiques and revises its own answers against written principles (SL stage), then RL from AI preference labels; scalable, auditable labeling — verify against humans.

TRAPS

- ▶ “LoRA $r=8$ is 8/4096 of params.” It is $r(d_{in}+d_{out})$ per matrix, summed over targets.
- ▶ “SFT loss went down so SFT worked” / “reward went up so RL worked.” Evaluate generations with a stricter grader.
- ▶ “A baseline biases the gradient.” Only if it depends on the sampled action.
- ▶ The vocabulary trap: Qwen2.5-0.5B logits for 4×1024 tokens are 2.5 GB in fp32 — more than its weights. Chunk the loss.

10 Reasoning models, post-training data, adaptation

[gap-fill · extends section on post-training]



INTERVIEW Q&A | REASONING AND TEST-TIME COMPUTE

Q162. What changed with o1 / R1-style models?

RL on verifiable rewards (math answers, unit tests) teaches long chains of thought with self-checking and backtracking; accuracy scales with both RL compute and the number of thinking tokens at inference.

Q163. Self-consistency vs best-of- N vs search?

Self-consistency: majority vote over sampled answers (no verifier, needs a final answer to vote on). Best-of- N : pick by a reward model or verifier; gains saturate and the RM gets exploited at large N . Search (beam over steps, MCTS) needs a PRM and is costly; sequential revision helps on easy problems, parallel sampling on hard ones.

Q164. How do you control thinking length?

Length penalties or budget tokens in RL; “budget forcing” at inference (append “Wait” to extend, force the answer token to stop, as in s1); train on a mix of short and long traces; route easy queries to no-think mode.

Q165. What is overthinking and how do you measure it?

Many tokens spent after the answer is already found, or on trivial problems. Measure accuracy vs tokens curves per difficulty bucket; tokens-to-first-correct; reward shaping on length within correct samples.

Q166. Is the visible chain of thought faithful?

Not necessarily: models can use hints without mentioning them. Treat CoT as useful evidence for monitoring, not ground truth; optimizing CoT for appearance can hide misbehavior (obfuscation).

Q167. Why distill reasoning into small models instead of RL on them?

SFT on a strong model’s traces transfers reasoning cheaply and beat RL directly on small models in R1’s report; RL on small models still adds on top once they have the behaviors.

Q168. What makes a good verifier?

Exact-match with normalization (math: sympy equivalence), sandboxed unit tests with hidden tests, format checks; guard against reward hacking (special-casing tests, printing expected output, editing the checker).

INTERVIEW Q&A | POST-TRAINING DATA

Q169. How do you build an SFT set?

Define the target behaviors and a taxonomy (tasks, domains, difficulty, safety); seed with expert-written examples; expand with

synthetic generation (Self-Instruct, Evol-Instruct, persona-driven) filtered by a judge and verifiers; dedup; decontaminate against evals; balance the mix; a few thousand excellent examples beat many mediocre ones (LIMA).

Q170. How do you collect preference data?

Sample 2+ responses per prompt from the current policy (on-policy matters), have raters or a judge pick with a rubric, record margins and ties, measure inter-annotator agreement, refresh iteratively (on-line/iterative DPO).

Q171. Annotation guidelines that work?

Concrete rubric with examples of each grade, tie-break rules, what to do with unsafe/unsure, calibration rounds, gold questions to monitor raters, disagreement adjudication.

Q172. Synthetic data risks?

Model collapse from recursive training on own outputs (keep real data in the mix), homogenized style, inherited errors and hallucinations, license/terms of the generator, contamination of evals.

Q173. How do you filter data quality at scale?

Heuristics (length, language ID, repetition), perplexity filters, a classifier trained on judge labels (FineWeb-Edu style), dedup (exact + MinHash), PII scrubbing; ablate the filter on small proxy models.

INTERVIEW Q&A | ADAPTING A MODEL TO YOUR DOMAIN

Q174. Continued pretraining vs SFT?

CPT on raw domain text (billions of tokens) teaches vocabulary and knowledge; SFT teaches the task format. Typical: CPT with a replay of general data (5–30%), lower LR with re-warmup, then SFT and preference tuning.

Q175. How do you prevent catastrophic forgetting?

Replay general data, lower LR, fewer steps, LoRA (forgets less, learns less), regularize toward the base (KL), evaluate general benchmarks alongside domain ones.

Q176. What is model merging and when does it work?

Average weights of fine-tunes from the same base (model soups), task arithmetic (add $\tau = \theta_{ft} - \theta_{base}$), TIES/DARE resolve sign conflicts and sparsify deltas. Works because fine-tunes stay in one loss basin; fails across different bases.

Q177. Your fine-tuned model got worse at following instructions. Why?

Wrong chat template or missing EOS, loss on prompt tokens, overfitting (too many epochs), low-diversity data, LR too high; check with a held-out instruction-following eval and compare to the base.

Q178. Fine-tuning a model to call your tools?

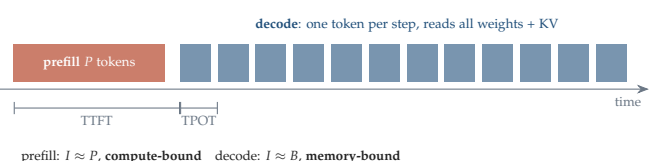
Collect real or synthetic multi-turn traces with tool calls, results and final answers in the provider’s tool format; include no-tool and error-recovery cases; mask loss on tool outputs; evaluate on exact argument match and end-task success.

STARTING HYPERPARAMETERS (THEN SWEEP)

full SFT (7–8B)	LR 1e-5–2e-5, cosine, warmup 3%, 2–3 epochs, batch 64–128 seqs, loss on responses only
LoRA SFT	LR 1e-4–2e-4, r 16–64, $\alpha = 2r$ (or rsLoRA), all linear layers, dropout 0–0.05
DPO	LR 5e-7–1e-6 (full) / 5e-6 (LoRA), $\beta = 0.1$, 1–2 epochs, watch chosen log-prob
GRPO	LR 1e-6, $G = 8–16$, temp 1.0, KL 0–0.04, clip 0.2/0.28, max length budgeted
CPT	0.1–0.3× pretrain peak LR, re-warmup, 5–30% replay

11 Inference and serving

[module 07 · labs 06 kernels, 07 KV cache + sampling, 13 quantization, 14 speculative decoding]



SERVING METRICS AND THE STEP-TIME MODEL

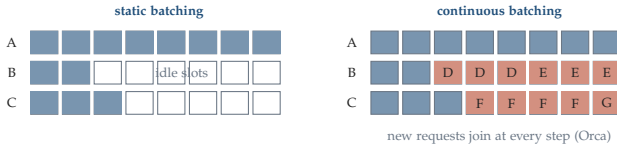
TTFT = queue + prefill. TPOT/ITL = decode step. E2E = TTFT + $(n_{out} - 1) \cdot TPOT$. Goodput = throughput meeting SLOs.

$$t_{\text{step}} \approx \max \left(\frac{2NB}{F_{\text{peak}}}, \frac{2N + B n_{\text{ctx}} s_{\text{kv}}}{\text{BW}} \right) + t_{\text{overhead}}$$

Llama-3-8B, H100: batch-1 decode 4.8 ms (≤ 210 tok/s). Batch 64 @ 2k ctx: KV 17.2 GB > weights 16 GB; step ≈ 9.9 ms, $\approx 6,500$ tok/s. Prefill 1k tokens: 16 TFLOP ≈ 16 ms at peak.

model (bf16)	KV/token	32k ctx	shape
Qwen2.5-0.5B	12 KiB	0.4 GiB	24L, 2 kv $\times$ 64
Llama-3-8B	128 KiB	4 GiB	32L, 8 kv $\times$ 128
Llama-3-70B	320 KiB	10 GiB	80L, 8 kv $\times$ 128

For any serving question write three numbers first: weight bytes, KV bytes/token, HBM per GPU.



INTERVIEW Q&A | SERVING SYSTEMS

Q179. Why is decode slow?

Not because it is sequential: each step reads every weight and the whole KV cache to make one token per sequence ($I \approx 1$). The fix is batching across users.

Q180. Continuous batching?

Re-form the batch every decode step: finished sequences leave, waiting ones join (Orca). Needs ragged/paged kernels and a per-step scheduler deciding what fits in memory.

Q181. PagedAttention?

KV in fixed blocks (16 tokens in vLLM) + per-sequence block table, like virtual memory. Waste $\leq$ one partial block per sequence (vs 60–80% before, <4% after) $\rightarrow$ bigger batches. Copy-on-write sharing for parallel samples; preempt by recompute or swap.

Q182. Prefix caching — and the timestamp trap?

Hash full KV blocks by their tokens + everything before (vLLM APC) or keep a radix tree (SGLang RadixAttention). Put stable parts first (system prompt, tools, docs); one timestamp at the top makes every request a miss. Route same-prefix requests to the same replica.

Q183. Chunked prefill vs disaggregation?

Chunked (Sarathi): fixed per-step token budget, decode tokens first then a prefill chunk — no stalls, prefill piggybacks on idle FLOPs. Disaggregated (DistServe, Mooncake): separate prefill and decode pools, ship KV over RDMA (70B: 320 KiB/token $\rightarrow$ 330 GB/s at 10^6 tok/s). Disaggregate at scale with strict TTFT and TPOT SLOs.

Q184. TP for serving?

Cuts per-token latency (each GPU streams 1/TP of weights) at the cost of 2 all-reduces per layer; use the smallest TP that fits weights + KV for your batch. Replicas (DP) for throughput.

Q185. What do CUDA graphs fix?

CPU launch overhead: hundreds of tiny kernels per small-batch decode step. Record once, replay; one graph per batch-size bucket (pad up). Big win at batch 1–8.

Q186. “Temperature 0 is deterministic”?

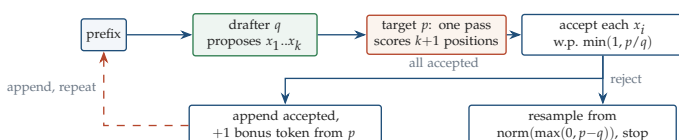
Not across runs: batch composition changes reduction orders. Needs batch-invariant kernels.

Q187. How does grammar-constrained decoding work?

Compile JSON schema/regex to an automaton over the vocab; mask disallowed logits each step; overlap mask computation with the forward. Guarantees it *parses*, not that it is correct.

SAMPLING

logits $/T \rightarrow$ top- k (keep k best) $\rightarrow$ top- p (smallest set with mass $\geq p$) $\rightarrow$ min- p (keep $p_i \geq p_{\min} \cdot p_{\max}$) $\rightarrow$ sample. $T \rightarrow 0$: greedy. Repetition/frequency/presence penalties edit logits of seen tokens. Beam search: high-likelihood, bland text; fine for translation/ASR, bad for open-ended chat.



SPECULATIVE DECODING

Exact: $q(x) \min(1, \frac{p}{q}) + (1 - \alpha) \frac{\max(0, p-q)}{1-\alpha} = \min(p, q) + \max(0, p - q) = p(x)$.

$\mathbb{E}[\text{tokens/round}] = \frac{1-\alpha^{k+1}}{1-\alpha}$; speedup = $\frac{1-\alpha^{k+1}}{(1-\alpha)(kc+1)}$, $c = \text{draft/target cost}$.

$\alpha=0.8, k=4, c=0.05$: 3.36 tokens/round, $2.8 \times$. α matters more than c .

Drafters: small same-family model, Medusa heads, EAGLE-1/2/3 (feature-level, highest acceptance), MTP modules (DeepSeek-V3), n-gram / prompt lookup (free for RAG, code edits), lookahead decoding.

INTERVIEW Q&A | SPECULATIVE DECODING AND KERNELS

Q188. Why is speculative decoding lossless?

The accept/resample rule makes every emitted token distributed exactly as p (identity above); rejected positions end the round. The greedy-match shortcut is exact only for greedy decoding. Bitwise equality is not promised (different kernels).

Q189. When does it hurt?

Large batches: decode nears the compute roof, verifying $k+1$ tokens is no longer free and rejected drafts waste FLOPs; drafter KV and memory; low-acceptance content (creative text). Engines reduce k or disable under load.

Q190. Why does FlashAttention win?

IO: naive attention writes/reads S and P ($T \times T$) through HBM (128 MiB/head at 8k). FA tiles Q, K, V in SRAM with **online softmax**, never materializing S/P : HBM traffic $\Theta(T^2 d^2 / M)$, extra memory $O(T)$. It adds FLOPs (recompute P in backward) and is faster.

Q191. Online softmax?

Per row keep running max m , sum ℓ , accumulator o . New tile: $m' = \max(m, \max s)$; $\ell \leftarrow e^{m-m'} \ell + \sum e^{s-m'}$; $o \leftarrow e^{m-m'} o + e^{s-m'} V$; divide by ℓ at the end. Save LSE = $m + \log \ell$ for backward.

Q192. FA-2 vs FA-3?

FA-2: fewer non-matmul FLOPs (normalize once), parallelize over sequence, better warp split. FA-3 (Hopper): warp specialization with TMA producers, softmax/matmul overlap, FP8 with Hadamard incoherent processing.

Q193. FlashDecoding?

At batch 1 one block per (seq, head) leaves most SMs idle; split the KV length into chunks across blocks, each yields partial (m, ℓ, o) , merge with a tiny kernel.

Q194. What else gets fused in decode?

RMSNorm + residual add, RoPE on Q/K, SwiGLU, dequant inside GEMM, sampling — each fusion removes an HBM round trip and a launch.

QUANTIZATION: $q = \text{clamp}(\text{round}(w/s) + z), \hat{w} = s(q - z)$

Symmetric $s = \max |w| / (2^{b-1} - 1)$; MSE $\approx s^2 / 12$; one outlier sets s for its whole group. INT4 with a 16-bit scale per 128 weights = 4.125 bits.

method	what	bits	outlier trick
RTN	W	8/4	finer groups only
LLM.int8()	W+A	8	outlier dims in fp16
GPTQ	W	4/3	column-wise, error to later cols via $H^{-1}, H = 2X^T X$
AWQ	W	4	scale up $\sim 1\%$ salient channels by act. magnitude
SmoothQuant	W8A8	8	$s_j = \frac{\max X_j ^a}{\max W_j ^{1-a}}$ moves scale $A \rightarrow W$
FP8 e4m3	W+A+KV	8	per-tensor/channel/block scales
NF4	W store	4	normal-quantile levels, block 64
QuaRot/SpinQuant	W+A+KV	4	Hadamard/learned rotations spread outliers
MX/NVFP4	W+A	4	block scales (32 pow2 / 16 fp8)
KIVI	KV	2	keys per-channel, values per-token
GGUF k-quants	W	2–8	super-blocks (llama.cpp)

Memory: 8B $\rightarrow$ 16 / 8 / 4.1 GB (bf16 / 8-bit / INT4); 70B $\rightarrow$ 141 / 71 / 36 GB.

INTERVIEW Q&A | QUANTIZATION

Q195. Which phase does weight-only INT4 speed up?

Memory-bound decode ($4\times$ fewer weight bytes, close to that speedup at small batch). Not compute-bound prefill (dequant even costs a bit); gain shrinks at large batch. W8A8/FP8/FP4 run the matmul in low precision and speed prefill too.

Q196. Why are activations harder than weights?

Weights are static and near-Gaussian; LLM activations have a few channels with huge magnitude (emerging around 6.7B, Dettmers), which wreck shared scales.

Q197. How do you evaluate a quantized model?

On your task, paired, with CIs — perplexity hides damage in long-context, math, code, non-English. Compare against a bf16 model of the *same memory*. Check which layers stayed high precision, group size, calibration data.

Q198. Why quantize the KV cache?

More sequences / longer context in the same HBM and fewer bytes per long-context decode step; FP8 KV is standard.

INTERVIEW Q&A | CAPACITY PLANNING: THE CLASSIC QUESTION

Q199. Serve Llama-3-70B at 1,000 req/s, 1,000 in / 300 out, p95 TTFT < 1 s, H100s. How many GPUs?

Model: 141 GB bf16; KV 320 KiB/token; 141 GFLOP/token. **Load:** prefill 10^6 tok/s, decode $3\cdot 10^5$ tok/s. **Prefill** (compute): 141 PFLOP/s $\div$ 495 TF/s (50% MFU) $\approx$ 285 GPUs, ~ 360 at 80% utilization; one prompt on TP8 = 36 ms, so TTFT is queueing. **Decode** (memory): TP8 replica has ~ 435 GB for KV; batch 256 $\rightarrow$ ~ 12 ms TPOT, 21k tok/s/replica $\rightarrow$ 15 replicas $\approx$ 150 GPUs. **Total** $\approx$ 510 H100s; at an assumed \$2.5/GPU-h $\approx$ \$1.2 per M output tokens. **Lever:** FP8 roughly halves the fleet; prefix caching cuts prefill by hit rate. **Risks:** long-prompt bursts, p95 output tails, KV transfer fabric, stragglers.

Q200. Same method on an 8 GB laptop GPU?

Qwen2.5-1.5B bf16 = 3.1 GB, ~ 3.5 GB left for KV at 28 KiB/token $\approx$ 130k tokens (64 chats of 2k). Batch-1 bound $3.1/0.448 \approx 6.9$ ms/token.

ENGINES (OCT 2026)

vLLM: PagedAttention, broad support, APC, spec decode, disagg; default OSS server + RL rollouts. **SGLang:** RadixAttention, fast structured output, agents. **TensorRT-LLM** (+ Dynamo): NVIDIA kernels, FP8/FP4. **llama.cpp**/Ollama: GGUF, CPU/Apple/consumer. **MLC:** phones, WebGPU.

TRAPS

- ▶ “INT4 makes everything $4\times$ faster.” Decode only, small batch.
- ▶ “FlashAttention reduces FLOPs.” It reduces HBM traffic and memory.
- ▶ “More GPUs per replica = more throughput.” TP adds all-reduces; more, smaller replicas often win per GPU.
- ▶ “KV is small next to weights.” Llama-3-8B, batch 64 @ 2k: KV 17 GB > weights 16 GB.

12 Evaluation: measure like a scientist

[module 08 · lab 15 eval stats]



every stage hides silent bugs: read 20 raw transcripts

STATISTICS YOU MUST QUOTE

95% CI of accuracy	$\hat{p} \pm 1.96 \sqrt{\hat{p}(1-\hat{p})/n}$; Wilson/bootstrap near 0, 1 or small n
half-widths	$n=100@50\%: \pm 9.8$; GPQA-D 198@70%: ± 6.4 ; 1000@70%: ± 2.8
paired	SE of mean of $d_i = a_i - b_i$; McNemar on discordant pairs
clustered	design effect $1 + (m-1)\rho$ ($m=5, \rho=0.3$: $2.2\times$ variance)
seeds	$t_{0.975, k-1}$: 4.30 for 3 seeds, not 1.96
power (unpaired)	70 vs 72%: 8,080 items/arm; 50 vs 60%: 388
pass@k (unbiased)	$1 - \binom{n-c}{k} / \binom{n}{k}$; $n=20, c=3, k=5$: 0.601 (plug-in 0.556 is biased)
pass ^k (τ -bench)	all k tries succeed: $\binom{c}{k} / \binom{n}{k}$ — what users feel
multiple tests	10 null ablations @0.05: 40% chance ≥ 1 “win”; Holm, BH
Cohen’s κ	$\frac{p_o - p_e}{1 - p_e}$; 85% raw with 80/20 marginals $\rightarrow \kappa=0.53$
judge correction	$\hat{p} = \frac{p_{\text{obs}} + \text{TNR} - 1}{\text{TPR} + \text{TNR} - 1}$
Elo	100 pts $\approx$ 64% win; 30 pts $\approx$ 54%

INTERVIEW Q&A | BENCHMARKS AND VALIDITY

Q201. What makes an eval valid?

Construct (measures the capability), content (covers the space), internal (difference caused by model, not harness), external (predicts your deployment), reliability (repeatable), freshness (not contaminated). Validity before precision.

Q202. Name today’s benchmark landscape.

Knowledge: MMLU (saturated), MMLU-Pro. Expert: GPQA Diamond, HLE. Math: GSM8K/MATH (saturated), AIME (fresh, tiny), FrontierMath. Code: HumanEval (saturated), LiveCodeBench, SWE-bench Verified (scaffold-dependent). IF: IFEval. Agents: τ -bench, GAIA, OSWorld, MLE-bench. Long tasks: METR time horizon (~ 7 -month doubling 2019–24). ARC-AGI. LMArena (style/length effects). And **your own product evals**, the only uncontaminated ones.

Q203. How do you detect contamination?

n -gram overlap (misses paraphrase), canary strings, Min-K% prob, exchangeability (canonical order more likely than shuffles), post-cutoff splits (LiveCodeBench dates items), fresh parallel sets (GSM1k), completion probes. Keep a private held-out split.

Q204. How do you calibrate an LLM judge?

Binary/3-level rubric; humans label 100–200 items (human–human agreement is the ceiling); report confusion matrix, TPR, TNR, κ on a held-out split. Mitigate position (swap order), verbosity (length-controlled), self-preference (different family), surface cues (grade specific claims).

Q205. Why is the harness part of the result?

Prompt format, few-shot choice, option order, extraction regex move scores by points. Fix and report it, compare under one harness, use the model’s chat template, give reasoning models enough tokens, report spread across 3 prompt variants.

INTERVIEW Q&A | STATISTICS

Q206. A is 72%, B is 70% on 1,000 items: 630 both right, 90 A-only, 70 B-only. Significant?

Unpaired CI on the difference ± 4.0 ; paired ± 2.5 (38% tighter); McNemar on 90 vs 70: $p \approx 0.13$. Not significant: only the 160 disagreements carry information.

Q207. 5 samples per question on 500 questions — what is n ?

500. Average within item first; samples on one item are correlated.

Q208. Minimum detectable effect on a 200-item eval?

Roughly 10 points unpaired. Say so before running.

Q209. How do you avoid fooling yourself with many ablations?

Explore on a dev split, pick a winner, run *one* confirmatory test on held-out data; otherwise Holm (FWER) or BH (FDR).

Q210. What goes in an eval report?

Model + version, harness, n items/clusters, score $\pm$ CI (method), paired difference $\pm$ CI, seeds and spread, variants tried, compute, per-item outputs path.

INTERVIEW Q&A | PRODUCT EVALS AND RESEARCH DISCIPLINE

Q211. How do you build evals for an LLM product?

Log real traces $\rightarrow$ read and open-code 100 $\rightarrow$ cluster into a failure taxonomy, count $\rightarrow$ 30–100 targeted items per failure mode (plus

passing regression guards) → graders: programmatic > reference-based > calibrated judge → run in CI on every prompt/model/retrieval change; gate on no significant regression. Put \$ and latency next to every quality number.

Q212. What makes an ablation survive review?

One change at a time, matched compute and tuning for baselines, multiple seeds, CIs, pre-registered metric, negative results reported, a reproduction script.

Q213. How do you compare models with different tokenizers on loss?

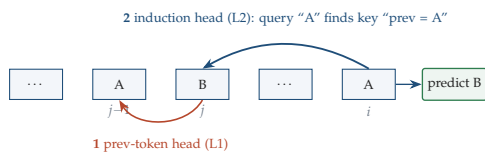
Bits per byte, never per-token loss.

TRAPS

- ▶ “+1.2 on MMLU, so better.” Ask for n , CI, pairing.
- ▶ Trusting 85% raw judge agreement: a constant “pass” judge gets 80%.
- ▶ “No contamination: we did n -gram decontam.” Paraphrases and translations pass.
- ▶ Claiming a win over a baseline that got less tuning or compute.

13 Interpretability, alignment, security

[module 09 - lab 16 interpretability]



K-composition: the key at j was written by head 1. Works for never-seen pairs, which is in-context copying.

INTERVIEW Q&A | MECHANISTIC INTERPRETABILITY

Q214. What is the residual-stream view?

Every layer reads from the stream (via a norm) and adds to it (via an out-projection); the stream is a shared linear communication channel, so contributions decompose additively into paths.

Q215. QK and OV circuits?

Per head, $W_Q W_K^T$ decides where to attend, $W_V W_O$ decides what is moved; each is a $d \times d$ matrix of rank $\leq d_h$. Only these products matter, not individual matrices.

Q216. Explain induction heads and why they matter.

Two-head circuit “... A B ... A → B” (diagram). They form in a sudden phase change early in training (visible bump in loss) coinciding with a jump in in-context learning (Olsson et al. 2022). Induction score: attention to $i - T + 1$ on period- T repeats.

Q217. What is superposition?

With sparse features the model stores more features than dimensions in almost-orthogonal directions and tolerates interference (ReLU + negative bias filter it). Hence **polysemantic neurons** are expected (Toy Models of Superposition).

Q218. Sparse autoencoders?

Overcomplete dictionary $F \gg d$: $f = \text{ReLU}((x - b_d)W_e + b_e)$, $\hat{x} = fW_d + b_d$, loss $\|x - \hat{x}\|^2 + \lambda \sum f_i \|W_{d,i}\|$. Variants: TopK (no shrinkage, sets L0), BatchTopK, Gated, JumpReLU (Gemma Scope), transcoders, cross-layer transcoders. Report L0, FVU, ΔCE in nats, dead latents, auto-interp — at matched L0.

Q219. SAE pitfalls?

Dead latents, feature splitting (no single true width), feature absorption, dark matter in the error, top-activation illusions; SAE probes didn’t beat plain linear probes downstream (Kantamneni 2025).

Q220. Probe gets 95%. Does the model use the feature?

Decodable $\neq$ used. Check selectivity vs control tasks and a random model; prefer difference-of-means directions; intervene (ablate/add) and check behavior changes as predicted.

Q221. Denoising vs noising patching?

Denoising (clean → corrupted run) tests *sufficiency*; noising (corrupted → clean) tests *necessity*. Metric: normalized logit difference $(m_p - m_c) / (m_{\text{clean}} - m_c)$. Path patching isolates edges (IOI circuit); attribution patching = first-order Taylor (cheap, breaks at saturation); ACDC automates. Beware self-repair (backup heads).

Q222. Steering?

Add $c \cdot v$ to the residual stream (v from contrastive means); directional ablation $x \leftarrow x - (x \cdot \hat{r})\hat{r}$ — one “refusal direction” mediates refusal in many open models (Arditi 2024). Evaluate collateral dam-

age and compare with just prompting.

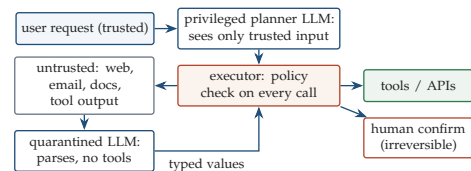
Q223. What are attribution graphs?

Replace MLPs by a cross-layer transcoder, freeze attention/norms for one prompt, add error nodes → linear local model; graph of direct effects between features; validate by intervening on the real model. Found two-hop reasoning, rhyme planning, multilingual features.

ALIGNMENT FAILURE MODES: MECHANISM + EVIDENCE

reward hacking	optimizer finds unintended high reward; coding agents edit tests (Baker 2025)
overoptimization	proxy up, gold down past a KL (Gao 2022)
sycophancy	raters prefer agreement; RLHF amplifies (Sharma 2023)
unfaithful CoT	stated reasoning $\neq$ computation (Turpin 2023)
obfuscation	penalizing bad thoughts hides them (Baker 2025)
alignment faking	behaves differently when it thinks it’s trained (Greenblatt 2024)
sleeper agents	backdoors survive SFT/RL/adversarial training (Hubinger 2024)
sandbagging	strategic underperformance on capability evals
in-context scheming	disables oversight in agentic setups (Apollo 2024)
emergent misalignment	narrow fine-tune (insecure code) → broad misbehavior (Betley 2025)

AI control: assume possible misalignment; trusted monitoring by a weaker model, auditing samples, restricted affordances.



INTERVIEW Q&A | SECURITY FOR LLM APPS AND AGENTS

Q224. Direct vs indirect prompt injection?

Direct: the user attacks your prompt. Indirect (Greshake 2023): anyone who controls text the model reads (web page, email, PDF, issue, tool result, MCP tool description) plants instructions executed with *your user’s* privileges. No in-band separation of instructions and data exists.

Q225. What is the lethal trifecta?

Private data access + exposure to untrusted content + an external communication channel → assume exfiltration. Remove one leg per task.

Q226. Defenses, strongest first?

(1) Architecture: plan-then-execute, dual LLM, CaMeL, context minimization. (2) Least privilege: scoped, per-tenant, read-only tokens; egress allowlist; spend caps. (3) Human confirmation showing the exact action. (4) Instruction hierarchy / StruQ training. (5) Injection classifiers, output filters (markdown image exfiltration). (6) Logging with provenance, fast revocation. Evaluate attack success *and* utility under attack (AgentDojo), with adaptive attacks.

Q227. Why do jailbreaks work?

Competing objectives (role-play, prefix injection) and mismatched generalization (base64, low-resource languages). Also GCG adversarial suffixes, many-shot jailbreaking. Defenses: safety training, input/output classifiers (Constitutional Classifiers), rate limits, monitoring.

Q228. Does a system prompt saying “ignore instructions in documents” work?

It blunts naive attacks, fails against adaptive ones. Not a defense.

Q229. What governance should you know?

Frontier safety frameworks with capability thresholds (Anthropic RSP / ASL levels, OpenAI Preparedness, Google DeepMind FSF), EU AI Act GPAI obligations, AI security institutes’ pre-deployment testing. Have a defended position, not slogans.

TRAPS

- ▶ “This head attends to X, so X matters.” Attention ignores value

norms and the OV map.

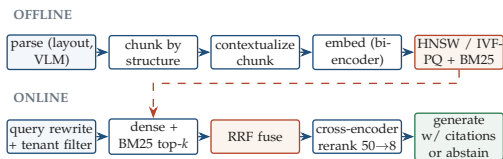
- ▶ “95% loss recovered” vs a zero-ablation baseline; report ΔCE .
- ▶ “Ablating the head did nothing.” Self-repair can hide it.
- ▶ “The CoT shows what the model thinks.” Not guaranteed faithful.

14 Applied LLM systems: RAG, agents, LLMOps

[module 10 - lab 17 retrieval]

WHICH TOOL FIRST?

behavior, tone, framing	prompting (cheapest; measure first)
changing / private / cited knowledge	RAG : update the index, not the model; ACL at retrieval
format, style, narrow skill	fine-tune (LoRA): teaches <i>how</i> , poor at injecting facts
small corpus, relaxed latency	long context + prompt caching
lower cost at fixed quality	distill / smaller model + RAG, once evals prove parity



INTERVIEW Q&A | RETRIEVAL-AUGMENTED GENERATION

Q230. Where is RAG quality lost first?

Parsing: PDFs are layout, not text (columns, tables, footnotes inter-leave). Use layout-aware parsers or VLMs; eyeball 20 parsed docs before building anything.

Q231. How do you chunk?

Along structure (sections, list items, table rows with headers); 200–800 tokens, 10–20% overlap as a start — a hyperparameter to sweep per question type. Parent-document retrieval: retrieve small, pass the enclosing section. Contextual retrieval: prepend a doc-specific summary to each chunk before embedding/BM25 (49% fewer top-20 failures, 67% with reranking).

Q232. Bi-encoder vs cross-encoder vs ColBERT?

Bi-encoder embeds query and doc separately (fast, pre-computed; first stage). Cross-encoder reads them together (accurate, one forward per pair; rerank top 50–100). ColBERT: token-level late interaction (MaxSim), higher recall, bigger index.

Q233. How are embedding models trained?

Contrastive InfoNCE on (query, positive) pairs with in-batch and hard negatives: $\mathcal{L} = -\log \frac{e^{s(q,d^+)}/\tau}{\sum_j e^{s(q,d_j^+)}/\tau}$. Bigger batches = more negatives. Matryoshka training makes prefixes usable (1024→256 dims, 4× smaller).

Q234. HNSW vs IVF-PQ?

HNSW: layered proximity graph, greedy descent; params M , efConstruction, efSearch; great recall/latency, memory-heavy, awkward deletes. IVF: k-means into $\sim \sqrt{N}$ cells, probe nprobe; IVF-PQ compresses to 64 B/vector for billion scale. Always measure ANN recall vs exact search.

Q235. Index memory for 1M chunks × 1024 dims?

fp32 4.1 GB; fp16 2.0; int8 1.0; binary 128 MB; PQ (64 B) 64 MB.

Q236. Why hybrid search, and how do you fuse?

Dense matches meaning; BM25 matches exact strings (IDs, error codes, rare names). Reciprocal rank fusion $\sum 1/(60 + \text{rank})$ needs no score normalization and rewards agreement.

Q237. BM25 in one line?

$\sum_{t \in q} \text{IDF}(t) \frac{f(t,d)^{k_1+1}}{f(t,d)^{k_1+1} + (1-b+b \cdot |d|/|d|)}, k_1 \approx 1.2, b \approx 0.75$: saturating term frequency, length normalization.

Q238. MMR?

$\arg \max_d \lambda \text{sim}(q,d) - (1-\lambda) \max_{s \in S} \text{sim}(d,s)$: relevance minus redundancy, for near-duplicate results.

Q239. Query-side tricks?

Rewriting (resolve pronouns, expand acronyms), decomposition for multi-hop, HyDE (embed a hypothetical answer), metadata filters from the query, routing between indexes. Each adds latency — add only if the eval says so.

Q240. How do you make generation faithful?

Passage ids; cite after every claim; abstain when unsupported; most relevant passages at start/end (Lost in the Middle); for high stakes quote-then-answer and verify quotes verbatim.

Q241. How do you evaluate RAG?

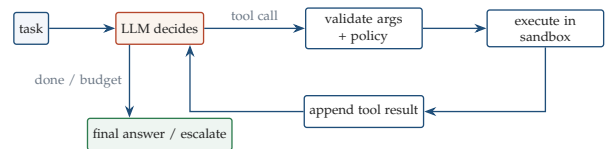
Separately then end-to-end. Retrieval: recall@ k (most important), MRR, nDCG. Generation: faithfulness, correctness, citation precision/recall, abstention. System: p50/p95 latency, cost/query. Eval set: 100–300 real questions, span-level labels (survive re-chunking), 10–20% unanswerable, multi-hop, exact IDs, every language; frozen test split; paired CIs.

Q242. Multi-tenant RAG security?

Filter by tenant/ACL *inside* the retrieval query, never post-filter generated text; per-tenant leak tests in CI; separate caches; retrieved docs are untrusted (injection).

RAG FAILURE → CHECK

doc exists, answer “not found”	retrieval miss: recall@ k , parse, chunk split
confident wrong + real citation	generator misread: faithfulness, quote-then-answer
entities mixed up	chunk lost context: contextualize, filters, MMR
IDs/codes never retrieved	dense-only: add BM25
drop after re-index	embedder/chunker changed; chunk-id labels
good offline, bad prod	eval unlike real queries: sample prod weekly



INTERVIEW Q&A | AGENTS AND TOOLS

Q243. What is an agent vs a workflow?

Agent: LLM in a loop choosing tool calls from observations until done or out of budget. Workflow: fixed code paths calling an LLM at set steps (chain, route, parallelize, orchestrator-workers, evaluator-optimizer). Prefer workflows when the path is known (Building Effective Agents).

Q244. Tool-calling mechanics?

Declare name + description (the model’s only docs) + JSON Schema; model emits structured calls (possibly parallel); runtime validates, checks policy, executes, appends result tied to call id; loop with step/token/time/cost budgets.

Q245. How do you design good tools?

Few, well-scoped, non-overlapping; descriptions say when *not* to use; concise outputs, pagination; actionable errors (“date must be YYYY-MM-DD; got 03/04/25”); idempotent where possible.

Q246. 95% per-step agent, 20 steps?

$0.95^{20} = 36\%$. $0.99 \rightarrow 82\%$. Verify-and-retry with recall r : failure $q \rightarrow q(1-r+rq)$; $q=0.05, r=0.8 \rightarrow 79\%$. pass⁸ at 90%/trial = 43%.

Q247. How do you sandbox an agent?

Container/microVM per task, egress off + allowlist, short-lived least-privilege tokens (never secrets in context), scoped FS with snapshots, CPU/mem/time/spend limits, human confirmation for irreversible actions, full audit log.

Q248. How do you evaluate an agent?

End state in resettable environments (not transcript claims), several trials (pass@1 with CL, pass^k), failure taxonomy from trajectories, cost/latency per task incl. failures, safety (injections planted). The scaffold is part of the system.

Q249. What is MCP?

Model Context Protocol: standard servers exposing tools, resources and prompts to any client. Transport and discovery, not security — descriptions and outputs are untrusted.

Q250. ReAct, reflection, planning?

ReAct interleaves reasoning and actions; Reflexion adds self-critique memory; plan-then-execute separates planning from untrusted data. Multi-agent only when subtasks are parallel and the eval shows a gain.

Q251. Memory for agents?

Short-term = context; compaction summaries; structured notes/memory files re-read later; vector or key-value long-term stores; sub-agents with clean contexts returning summaries. Evaluate recall after N turns, updates, cross-user leakage.

INTERVIEW Q&A | PROMPTING, CONTEXT ENGINEERING, STRUCTURED OUTPUT

Q252. Prompting techniques that matter?

Clear role + task + constraints + output format; few-shot examples (diverse, consistent format); CoT / “think step by step” for reasoning (less needed for reasoning models); decomposition and prompt chaining; self-consistency; delimiting untrusted data (XML tags) — helpful but not a security boundary.

Q253. What is context engineering?

Deciding what goes in the window, in what order, at what cost: stable first (system, tools, few-shot) for caching; long docs before the question, restate key instruction at the end; less irrelevant context (quality drops long before the limit); compaction for long horizons.

Q254. Constrained decoding: guarantees and costs?

Guarantees syntax, not semantics; can hurt reasoning when format fights the model — put a free-text reasoning field first, shallow schemas, enums; validate and retry with the error, log retry rate; version schemas like APIs.

LLMOPS AND COST

Cost/request = $T_{in,uncached}p_{in} + T_{in,cached}p_{cache} + T_{out}p_{out}$. Example (\$3 / \$0.30 / \$15 per M): 4k cached + 2k fresh + 500 out → \$0.0147 vs \$0.0255 uncached; output is now the biggest line.

Lever: prompt caching, shorter outputs, routing/cascades (FrugalGPT, RouteLLM), fewer better passages, batch APIs (~50% off), self-hosting at high utilization. Report **cost per successful task**.

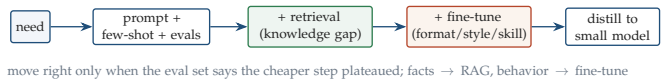
Ops: trace every call (prompt/config version, model, tokens cached/uncached, TTFT, cost, tools, tenant; OpenTelemetry GenAI spans); version prompt+tools+retrieval+model together; evals in CI; pin model versions, canary, rollback; fallback providers with circuit breakers. Semantic caching is risky (“cancel my order” ≈ “cancel my subscription”).

TRAPS

- ▶ “We switched embedding models and RAG improved” — with no recall@k, CI or pairing.
- ▶ Measuring RAG only end to end; post-filtering tenants after generation.
- ▶ Grading agents from their own transcript claims.
- ▶ “Constrained decoding means correct.” It means it parses.
- ▶ A timestamp or user name at the top of the system prompt kills prompt caching.

15 Shipping GenAI in production

[module 10 applied LLM systems · gap-fill for GenAI engineer loops]



INTERVIEW Q&A | BUILDING THE APP

Q255. Prompt vs RAG vs fine-tune, in one answer?

Prompting first (cheap, reversible). RAG when the model lacks knowledge that changes or must be cited. Fine-tune when you need a consistent format, tone, a narrow skill, lower latency or a smaller model; it teaches behavior, not reliable new facts. Distill once traffic justifies it.

Q256. Structure of a production prompt?

Role and goal; hard rules; tool descriptions; retrieved context in delimited blocks with ids; few-shot examples covering edge cases; output schema. Static parts first (prefix caching), dynamic parts last. Version prompts like code and gate changes on the eval set.

Q257. How do you get reliable JSON?

Native structured outputs / JSON-schema constrained decoding (grammar-masked logits) > tool calling > prompt + parse + retry. Keep schemas shallow, enums explicit, and let the model reason in a field *before* the answer field, since forcing format early can cost accuracy.

Q258. Guardrails stack?

Input: auth, rate limits, PII redaction, injection/jailbreak classifiers, topic filters. Output: schema validation, moderation classifier,

groundedness check, policy rules, PII scan. Tools: least privilege, allow-lists, human approval for irreversible actions. Log every decision.

Q259. Streaming: why and how?

Users perceive TTFT, not total latency. SSE/WebSockets token streams; buffer partial JSON with an incremental parser; moderate on chunks with a short lag; allow cancel to stop generation and save cost.

Q260. What should you cache?

Provider prompt caching (shared prefix, often ~90% cheaper input), exact-match response cache (keyed on model + prompt + params), semantic cache (embedding similarity; risky for personalized or time-sensitive answers), retrieval and embedding caches.

Q261. Latency budget for a voice agent?

Target <800 ms to first audio: VAD/endpointing ~200, STT final ~150, LLM TTFT ~250, TTS first chunk ~150. Stream every stage, allow barge-in, keep turns short, or use a native speech-to-speech model.

Q262. Long context or RAG?

Long context: simpler, good for one big document, but cost ∝ tokens, slower TTFT and mid-context recall drops. RAG: cheaper per query, citable, scales to corpora, fails on retrieval misses. Common hybrid: retrieve generously, then let a long-context model read.

Q263. Choosing an embedding model?

Evaluate on your queries (recall@k on 100–500 labeled pairs), not MTEB rank. Check max length, languages, dims (Matryoshka truncation), query/document prefixes, cost and license. Re-embedding the corpus is the migration cost; version indexes.

Q264. Multi-model routing?

Classifier or small LLM picks cheap vs strong model by difficulty; cascade (cheap first, escalate on low confidence or failed validation). Measure quality per route; routers drift with traffic.

Q265. How do you handle PII and data residency?

Classify data, redact before logging, zero-retention API terms or self-host, region-pinned endpoints, encryption, tenant isolation in indexes and caches, deletion paths (including embeddings and fine-tuning sets).

COST AND LATENCY LEVERS

cost driver	lever
input tokens	trim context, rerank to top- k , prompt caching, summarise history
output tokens	concise formats, <code>max_tokens</code> , stop sequences, JSON not prose
model size	route/cascade, distill, quantize (self-hosted)
utilization (self-host)	continuous batching, autoscaling, batch API for offline jobs (~50% off)
TTFT	shorter prompts, prefix caching, smaller model, regional endpoints
TPOT	speculative decoding, quantization, smaller model, less load
retries	validation + targeted repair instead of full regeneration

INTERVIEW Q&A | OPERATING IT

Q266. What do you monitor?

Latency (TTFT, TPOT, p50/p95/p99), error and refusal rates, tokens and cost per request/tenant, cache hit rate, tool-call failure rate, guardrail triggers, retrieval hit/empty rates, user feedback, and sampled LLM-judge quality scores. Traces per request (OpenTelemetry GenAI spans).

Q267. Offline vs online evaluation?

Offline: frozen golden set + regression suite on every prompt/model change (CI gate). Online: A/B tests on task success, thumbs, retention, escalation rate; shadow traffic for model swaps; canary rollouts with automatic rollback.

Q268. LLM-as-judge done right?

Rubric with anchored levels, pairwise > absolute, randomize position, hide model names, calibrate against 100–200 human labels (report agreement / κ), different family than the generator, re-validate after judge updates.

Q269. A provider silently updates the model. Protection?

Pin dated model versions, run the regression suite on schedule, alert on metric drift, keep a fallback provider/model behind the same interface.

Q270. How do you build the first eval set?

Collect 50–200 real or realistic inputs, include failure cases from logs, write expected properties (not exact strings), label with domain experts, split by difficulty and slice (language, tenant, intent), grow it from every incident.

Q271. Incident: answers suddenly cite wrong documents. Debug path?

Check recent changes (index rebuild, embedding model, chunker, prompt), retrieval logs for the failing queries, tenant filters, cache keys; reproduce offline, add the cases to the regression set, roll back first, fix second.

GENAI ENGINEER ANTI-PATTERNS

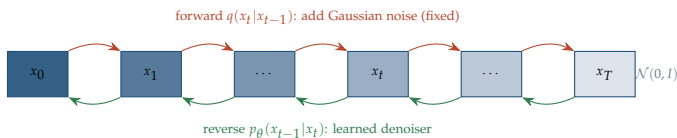
No eval set (“it looks good”); judging by a handful of demos; one giant prompt nobody owns; retries without validation; caching personalized answers; logging raw PII; letting tools act on untrusted text without approval; framework lock-in on the hot path; optimizing cost before quality is measured.

16 Generative AI beyond text: images, multimodal

[gap-fill beyond the repo · module 04 §8 touches ViT, CLIP, LLaVA]

GENERATIVE MODEL FAMILIES

Autoregressive	$p(x) = \prod_t p(x_t x_{<t})$; exact likelihood; slow sequential sampling (LLMs, VQ image tokens)
VAE	maximize ELBO $\mathbb{E}_{q(z x)} [\log p(x z)] - \text{KL}(q(z x) \ p(z))$; reparameterize $z = \mu + \sigma\epsilon$; blurry samples
GAN	$\min_G \max_D \mathbb{E} \log D(x) + \mathbb{E} \log(1 - D(G(z)))$; sharp, one-step; unstable, mode collapse
Diffusion	learn to reverse a noising process; stable, diverse, high quality; many steps
Flow matching	regress a velocity field along paths noise → data; straight paths → few steps (SD3, Flux)
Normalizing flows	invertible maps, exact likelihood via change of variables; constrained architectures



DIFFUSION IN FIVE LINES (DDPM)

Closed-form forward: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, $\bar{\alpha}_t = \prod_s (1 - \beta_s)$.
Loss (simple): $\mathbb{E}_{t, x_0, \epsilon} \|\epsilon - \epsilon_\theta(x_t, t)\|^2$ (predict the noise; or x_0 , or v).
 ϵ -prediction α score: $\nabla_x \log p_t(x) \approx -\epsilon_\theta / \sqrt{1 - \bar{\alpha}_t}$.
CFG: $\tilde{\epsilon} = \epsilon_\theta(x_t, \emptyset) + w(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \emptyset))$, $w \approx 3-8$; train with 10–20% condition dropout.
Flow matching: $x_t = (1 - t)x_0 + t\epsilon$, regress $v_\theta(x_t, t) \approx \epsilon - x_0$; sample by integrating the ODE.

INTERVIEW Q&A | DIFFUSION AND IMAGE GENERATION

Q272. Why does predicting noise work?

The noising has a closed form, so any (x_0, t) gives a supervised target. Denoising score matching: predicting ϵ is predicting the score, and the reverse SDE/ODE turns scores into samples. Training is a simple MSE — stable, unlike GANs.

Q273. DDPM vs DDIM sampling?

DDPM: stochastic ancestral sampling, ~1000 steps. DDIM: deterministic non-Markovian sampler with the same trained model, 20–50 steps, invertible (useful for editing). Further: DPM-Solver, distillation (consistency models, LCM, adversarial distillation) to 1–4 steps.

Q274. What is classifier-free guidance and its trade-off?

Train one model conditional and unconditional (drop the prompt sometimes); extrapolate toward the conditional prediction by w . Higher w : better prompt adherence and fidelity, less diversity, over-saturation. Costs 2 forward passes per step.

Q275. What is latent diffusion?

Run diffusion in a VAE’s latent (e.g. $64 \times 64 \times 4$ for 512^2 images, 48× fewer values) and decode at the end; text conditioning via cross-

attention to a text encoder (CLIP/T5). Stable Diffusion.

Q276. U-Net vs DiT?

U-Net: conv encoder–decoder with skips + attention. DiT: transformer over latent patches with adaLN conditioning on t and class/–text; scales cleanly with compute (SD3, Flux, Sora-style video use transformers + flow matching).

Q277. Noise schedules?

Linear/cosine β_t ; must reach near-pure noise at T (zero terminal SNR) or the model never learns to generate from true noise. Higher resolution needs shifted schedules (more noise).

Q278. How do you evaluate image generators?

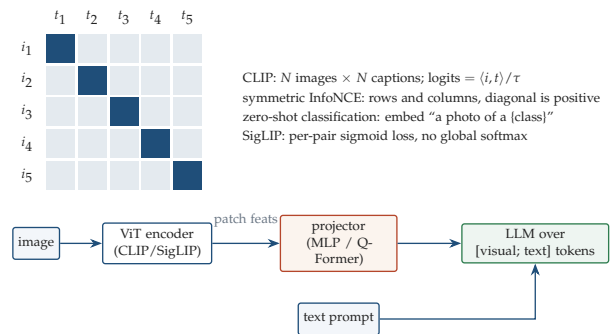
FID (Fréchet distance between Inception features of real and generated sets; lower is better, biased by sample size), CLIP score (text–image alignment), precision/recall for fidelity vs diversity, human preference (GenEval, arenas).

Q279. GAN failure modes?

Mode collapse, training instability, vanishing gradients when D wins. Fixes: non-saturating loss, Wasserstein + gradient penalty, spectral norm, two-timescale LR. Still used as decoders/discriminators in distillation and audio.

Q280. Fine-tuning diffusion models?

LoRA on attention, DreamBooth (subject, with prior preservation), textual inversion (learn a token embedding), ControlNet (zero-initialized copy of the encoder conditioned on edges/depth/pose), IP-Adapter (image prompts).



INTERVIEW Q&A | VISION, MULTIMODAL, SPEECH

Q281. How does a ViT tokenize an image?

Split into $P \times P$ patches (16×16), flatten, linear-project to d , add position embeddings (+ CLS token); a 224^2 image is 196 tokens. Weak inductive bias → needs lots of data or distillation.

Q282. How is a VLM like LLaVA built and trained?

Frozen vision encoder → projector → LLM. Stage 1: train the projector on image–caption pairs (alignment). Stage 2: instruction-tune projector + LLM on visual chat data. Variants: Q-Former (BLIP-2) compresses to fixed queries; cross-attention layers (Flamingo); native early fusion (Chameleon-style, all modalities as tokens).

Q283. Main cost knob in VLMs?

Number of visual tokens: high-res images tiled into crops (AnyRes) multiply tokens; pooling/resampling trades detail for context and latency.

Q284. How does Whisper work?

Encoder–decoder transformer on log-Mel spectrograms (30 s windows), trained on 680k h weakly supervised multilingual audio; special tokens select task (transcribe/translate), language, timestamps.

Q285. Contrastive vs generative pretraining for vision?

Contrastive (CLIP) gives aligned embeddings for retrieval/zero-shot; masked autoencoding (MAE) and generative objectives learn denser features; VLMs mostly reuse contrastive encoders.

Q286. Multimodal RAG?

Embed pages as images (ColPali-style late interaction over page patches) or caption/OCR then embed text; keep figures and tables as first-class chunks.

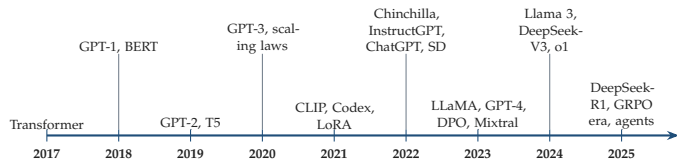
TRAPS

- ▶ “Diffusion is slow, period.” Distillation and flow matching reach 1–4 steps.
- ▶ “Higher CFG is better.” It trades diversity and saturation for adherence.
- ▶ Comparing FID across different sample counts or feature extractors.

- Ignoring visual-token count when sizing VLM latency and KV cache.

17 Model timeline and the tooling ecosystem

[gap-fill - snapshot October 2026; verify versions before quoting]



MILESTONES AND WHAT EACH PROVED

Transformer (2017)	attention alone, parallel training
GPT-1 / BERT (2018)	generative vs masked pretraining + fine-tuning
GPT-2 (2019)	pre-shot multitask from scale; pre-norm
T5 (2019)	everything as text-to-text
GPT-3 (2020)	in-context few-shot learning at 175B
Kaplan (2020) / Chinchilla (2022)	power laws; then 20 tokens/param
Codex (2021)	code LMs, pass@k
InstructGPT (2022)	SFT + RM + PPO; 1.3B preferred over 175B
LLaMA (2023)	open weights, over-trained small models
DPO (2023)	preference learning without RL
Mixtral (2023), DeepSeek-V3 (2024)	sparse MoE at quality; MLA, FP8
o1 (2024), DeepSeek-R1 (2025)	RL on verifiable rewards → long CoT reasoning

TOOLING MAP

modeling	PyTorch, JAX; HF transformers, tokenizers, datasets
fine-tuning	HF PEFT, TRL (SFT/DPO/GRPO), Axolotl, Unsloth, torchtune, LLaMA-Factory
RL at scale	verl, OpenRLHF, NeMo-RL; vLLM/SGLang as rollout engines
pretraining	Megatron-LM, DeepSpeed (ZeRO), PyTorch FSDP2, torchtitan, nanotron
kernels	Triton, CUDA, CUTLASS, FlashAttention, Liger kernels, torch.compile
serving	vLLM, SGLang, TensorRT-LLM, TGI, llama.cpp/Ollama, MLC; NVIDIA Dynamo
quantization	bitsandbytes, AutoGPTQ/GPTQModel, AutoAWQ, llm-compressor, GGUF
retrieval	FAISS, pgvector, Qdrant, Milvus, Weaviate, Elasticsearch/OpenSearch (BM25)
orchestration	LangChain/LangGraph, LlamaIndex, DSPy, provider SDKs, MCP
evals	lm-eval-harness, HELM, Inspect, OpenAI evals, RAGAS, promptfoo
observability	Langfuse, LangSmith, Phoenix, OpenTelemetry GenAI
experiment tracking	Weights & Biases, MLflow, TensorBoard

INTERVIEW Q&A | ECOSYSTEM QUESTIONS

Q287. Open-weight vs API model — how do you choose?

API: best quality, no infra, fast iteration, data-sharing terms. Open: control, privacy/on-prem, fine-tuning freedom, cost at high utilization, latency control. Decide with your eval set and a cost model at expected volume; keep the app model-agnostic.

Q288. When is a framework (LangChain etc.) worth it?

Prototyping and integrations. For production hot paths, thin provider SDK calls + your own orchestration are easier to debug, test and trace. DSPy when you want prompts optimized against a metric.

Q289. Which vector store?

Start with what you already run (Postgres + pgvector, OpenSearch); dedicated stores (Qdrant, Milvus) for scale, filtering performance, multi-tenancy; FAISS for in-process/offline. Choose by recall at your

latency, filtering, ops burden — not benchmarks.

Q290. HF generate vs vLLM?

generate is fine for experiments; vLLM/SGLang give continuous batching, paged KV, prefix caching — often 10×+ throughput for serving and RL rollouts.

Q291. What is torch.compile doing?

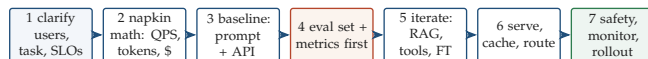
TorchDynamo captures graphs from Python bytecode, Inductor generates fused Triton kernels; mode="reduce-overhead" adds CUDA graphs. Wins on memory-bound elementwise chains and launch overhead.

Q292. Model licenses?

Apache-2.0/MIT (permissive) vs custom community licenses with use restrictions or user caps; also check output-use terms for synthetic data generated from API models.

18 System design: answer templates

[tracks/research-engineer: ml-system-design, interview-loops]



THE 7-STEP ANSWER (SAY THE STEPS OUT LOUD)

- Clarify:** users, task, inputs/outputs, languages, quality bar, latency (TTFT/E2E), cost per task, privacy/tenancy, failure cost.
- Napkin math:** requests/s, tokens in/out, context size, KV and GPU count or API \$/month.
- Simple baseline:** frontier API + prompt; it sets the quality ceiling and the cost floor.
- Evals before architecture:** real traces, failure taxonomy, graders, CIs, gates in CI.
- Iterate by failure bucket:** knowledge → RAG; actions → tools/agent; format/style/cost → fine-tune or distill.
- Serve:** model routing, prompt caching, batching, streaming, fallbacks.
- Safety and ops:** injection architecture, PII, guardrails, tracing, canary + rollback, feedback loop into evals.

INTERVIEW Q&A | WORKED DESIGNS

Q293. Design a support chatbot over 50k help-center docs for 10 tenants.

SLO: TTFT <1.5 s, answer with citations or escalate. **Index:** layout-aware parse, structure chunks + contextual headers, hybrid dense+BM25, tenant ACL as pre-filter, cross-encoder rerank to 8. **Gen:** mid-size model, cite-or-abstain, quote-then-answer for policy questions, structured output with escalate flag. **Eval:** 300 real tickets, span labels, 15% unanswerable, recall@10 + faithfulness + resolution rate, paired CIs. **Ops:** prompt caching (stable prefix), router to a small model for FAQs, per-tenant leak tests, injection tests with poisoned docs, human handoff. **Metric that matters:** resolved-without-escalation at fixed CSAT, cost per resolved ticket.

Q294. RAG vs fine-tuning for a legal-domain assistant?

Facts change and must be cited → RAG. Fine-tune (LoRA) only for format/terminology or to distill a cheaper model once evals show parity. Often both: small fine-tuned model that cites retrieved passages.

Q295. Design a code-review agent for PRs.

Inputs: diff + repo context (retrieval over symbols, tests). Tools: read file, grep, run tests in sandbox (no network, scoped tokens). Output: structured findings with file:line, severity, evidence. Eval: historical PRs with known bugs (recall) + false-positive rate judged by maintainers; pass^k consistency. Guard: never auto-merge; treat PR text as untrusted (injection).

Q296. Moderate 1B messages/day with an LLM?

Too costly per message on a frontier model (~10⁹ × tokens). Label a few hundred k with the frontier model + human audit, distill into a small classifier (encoder or 1B LLM), cascade: cheap model for all, frontier for uncertain (~1–5%), humans for appeals; calibrate thresholds per policy, monitor drift.

Q297. Design LLM serving for an internal 70B model.

Use the method of section 11: weights 141 GB, KV 320 KiB/token, TP8 replicas, continuous batching + paged KV + prefix caching, FP8, chunked prefill or disaggregation at scale, autoscale on queue depth, SLO-aware admission control, canary model rollouts.

Q298. How would you reduce hallucinations?

Ground with retrieval + citations + abstention; constrain outputs;

verifier pass (claims vs sources); lower temperature for factual tasks; train/prompt to say “I don’t know”; measure faithfulness and unsupported-claim rate on a labeled set.

Q299. How would you cut cost 5× without losing quality?

Measure per-slice quality first; prompt caching; trim context; shorter outputs; route easy traffic to a small model; batch offline work; distill the hot path; self-host at high utilization. Gate each change on paired evals.

Q300. Fine-tune a 7B model for a narrow task — end-to-end plan?

Baseline evals; 1–5k clean examples with the correct chat template and loss masking; QLoRA/LoRA r 16, all linears, LR $\sim 1\text{--}2\text{e-}4$, 1–3 epochs; eval each checkpoint with paired CIs incl. a general set for forgetting; optional DPO on own samples; quantize and serve; monitor drift.

INTERVIEW Q&A | RESEARCH-ENGINEER AND BEHAVIORAL

Q301. “Implement X from a blank file” — what do they expect?

Attention with causal mask + KV cache, RoPE, RMSNorm, top- p sampling, BPE merges, LoRA layer, DPO/GRPO loss, online softmax, AdamW step, beam search. Shape comments, test with tiny sizes, check against a reference (torch functional) and invariants.

Q302. Debug a diverging run in the interview?

Read the curves (section 5 table); check init loss vs $\ln V$; overfit one batch; inspect data; per-layer grad norms and max logits; bisect recent changes.

Q303. Tell me about a project (STAR + numbers).

Situation/Task in one line each; Action: what *you* did and why that option; Result: metric $\pm$ CI, cost, what broke, what you’d do next. Interviewers believe “I measured X” over “I built X”.

Q304. What paper influenced you most and what are its limits?

Pick one you’ve implemented (Chinchilla, DPO, FlashAttention, induction heads); state the claim, the experiment, a weakness, a follow-up you’d run.

TRAPS

- ▶ Jumping to fine-tuning or agents before a prompt baseline and an eval set.
- ▶ Designs without numbers (QPS, tokens, \$, GPUs) or without a rollback plan.
- ▶ Ignoring the long tail: p95 output length, long-prompt bursts, multi-lingual users.

19 Implement from scratch: code you write blind

[labs 01–18 · write each in under 10 minutes, then test against torch]

Every snippet is minimal PyTorch: shapes in comments, no error handling. Invariants to test are listed under each. Interviewers watch for: masks, shifts by one, dtype, and the dimension you reduce over.

1. Causal self-attention (multi-head, GQA-ready)

```
def attention(x, Wq, Wk, Wv, Wo, H, Hkv):
    B, T, d = x.shape; dh = d // H; g = H // Hkv
    q = (x @ Wq).view(B, T, H, dh).transpose(1, 2) # B,H,T,dh
    k = (x @ Wk).view(B, T, Hkv, dh).transpose(1, 2) # B,Hkv,T,dh
    v = (x @ Wv).view(B, T, Hkv, dh).transpose(1, 2)
    k = k.repeat_interleave(g, dim=1); v = v.repeat_interleave(g, dim=1)
    s = (q @ k.transpose(-2, -1)) / dh**0.5 # B,H,T,T
    mask = torch.ones(T, T, dtype=torch.bool).tril()
    s = s.masked_fill(~mask, float('-inf'))
    o = s.softmax(-1) @ v # B,H,T,dh
    return o.transpose(1, 2).reshape(B, T, d) @ Wo
```

Test: equals $F.scaled_dot_product_attention(is_causal=True)$; output at t unchanged when tokens $> t$ change.

2. RMSNorm and SwiGLU

```
def rmsnorm(x, g, eps=1e-6):
    return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + eps) * g
def swiglu(x, Wg, Wu, Wd):
    return (F.silu(x @ Wg) * (x @ Wu)) @ Wd
```

3. RoPE (interleaved pairs)

```
def rope(x, pos, base=10000.0):
    # x: ...,T,dh ; pos: T
    dh = x.shape[-1]
    inv = base ** (-torch.arange(0, dh, 2).float() / dh) # dh/2
    ang = pos[:, None].float() * inv[None] # T,dh/2
    cos, sin = ang.cos(), ang.sin()
    x1, x2 = x[..., 0::2], x[..., 1::2]
    out = torch.stack([x1*cos - x2*sin, x1*sin + x2*cos], -1)
    return out.flatten(-2)
```

Test: norm preserved; $(rope(q,m), rope(k,n))$ depends only on $m - n$.

4. Decode loop with KV cache + top- p sampling

```
def top_p_sample(logits, T=0.8, p=0.9):
    probs = (logits / T).softmax(-1)
    sp, idx = probs.sort(-1, descending=True)
    keep = sp.cumsum(-1) - sp < p # always keeps the top token
    sp = sp * keep; sp = sp / sp.sum(-1, keepdim=True)
    return idx.gather(-1, torch.multinomial(sp, 1))

@torch.no_grad()
def generate(model, ids, n_new):
    cache = None
    logits, cache = model(ids, cache=cache) # prefill
    for _ in range(n_new):
        nxt = top_p_sample(logits[:, -1])
        ids = torch.cat([ids, nxt], 1)
        logits, cache = model(nxt, cache=cache) # 1 token/step
    return ids
```

Test: cached and uncached generation give identical logits (greedy).

5. Online softmax (the FlashAttention core, one query row)

```
def online_attn(q, K, V, block=64):
    # q: dh ; K,V: T,dh
    m, l = float('-inf'), 0.0; o = torch.zeros_like(V[0])
    for s0 in range(0, K.shape[0], block):
        s = K[s0:s0+block] @ q / q.numel()**0.5
        m_new = max(m, s.max().item())
        scale = math.exp(m - m_new)
        p = (s - m_new).exp()
        l = l * scale + p.sum().item()
        o = o * scale + p @ V[s0:s0+block]
        m = m_new
    return o / l
```

6. Byte-level BPE training

```
def train_bpe(text, vocab_size):
    ids = list(text.encode('utf-8')); merges = {}
    for new in range(256, vocab_size):
        pairs = Counter(zip(ids, ids[1:]))
        if not pairs: break
        best = max(pairs, key=pairs.get)
        merges[best] = new
        out, i = [], 0
        while i < len(ids):
            if i < len(ids)-1 and (ids[i], ids[i+1]) == best:
                out.append(new); i += 2
            else:
                out.append(ids[i]); i += 1
        ids = out
    return merges # encode: apply merges in rank order
```

7. LoRA linear layer

```
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r=16, alpha=32):
        super().__init__(); self.base = base.requires_grad_(False)
        self.A = nn.Parameter(torch.empty(r, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, r))
        nn.init.kaiming_uniform_(self.A, a=5**0.5); self.s = alpha / r
    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.s
```

Test: output equals base at init; merged $W_0 + sBA$ gives identical outputs.

8. Sequence log-prob, DPO loss, GRPO advantages

```
def seq_logprob(logits, ids, resp_mask):
    # B,T,V ; B,T ; B,T
    lp = logits[:, :-1].log_softmax(-1) # predicts t+1
    tok = lp.gather(-1, ids[:, 1:, None]).squeeze(-1)
    return (tok * resp_mask[:, 1:]).sum(-1) # sum over response

def dpo_loss(pw, pl, rw, rl, beta=0.1):
    # policy/ref logits
    h = (pw - rw) - (pl - rl)
    return -F.logsigmoid(beta * h).mean()

def grpo_adv(r, eps=1e-6):
    # r: n_prompts, G
    mu, sd = r.mean(1, keepdim=True), r.std(1, keepdim=True)
    return (r - mu) / (sd + eps)

def clip_obj(logp, logp_old, A, e=0.2):
    rho = (logp - logp_old).exp()
    return torch.min(rho*A, rho.clamp(1-e, 1+e)*A).mean()
```

9. AdamW step

```
def adamw_step(p, g, m, v, t, lr, b1=0.9, b2=0.95, eps=1e-8, wd=0.1):
    p.mul_(1 - lr * wd) # decoupled decay
    m.mul_(b1).add_(g, alpha=1 - b1)
    v.mul_(b2).addcmul_(g, g, value=1 - b2)
    mh = m / (1 - b1**t); vh = v / (1 - b2**t)
    p.addcdiv_(mh, vh.sqrt()).add_(eps, value=-lr)
```

10. InfoNCE for embeddings, and MoE top- k routing

```
def info_nce(q, d, tau=0.05):
    # q,d: N,D normalized
    logits = q @ d.T / tau # in-batch negatives
    return F.cross_entropy(logits, torch.arange(len(q)))

def route(x, Wr, k=2):
    # x: N,d
    probs = (x @ Wr).softmax(-1) # N,E
    g, idx = probs.topk(k, -1)
    return g / g.sum(-1, keepdim=True), idx # renormalized gates
```

11. Speculative-decoding acceptance (one position)

```
def accept(x, p, q):
    # p, q: target/draft probs over V
    if torch.rand(()) < min(1.0, (p[x] / q[x]).item()):
        return x, True
    resid = (p - q).clamp(min=0); resid /= resid.sum()
    return torch.multinomial(resid, 1).item(), False
```

12. pass@k and a bootstrap CI

```
def pass_at_k(n, c, k):
    if n - c < k: return 1.0
    return 1 - np.prod(1 - k / np.arange(n - c + 1, n + 1))

def boot_ci(x, B=10_000, a=0.05):
    # per-item scores
    idx = np.random.randint(0, len(x), (B, len(x)))
    return np.quantile(x[idx].mean(1), [a/2, 1-a/2])
```

20 Worked napkin problems

[lab 03 napkin math · question bank §14 · two minutes each, assumptions out loud]

THE RECIPE

1 write the formula (FLOPs = $2N$ per token forward, $6N$ train; bytes = params $\times$ bytes/param; KV = $2LH_{kv}d_h$ ·bytes per token). 2 plug round numbers (H100: 989 TF/s bf16, 3.35 TB/s, 80 GB). 3 take the max of compute time and memory time. 4 apply a utilization (MFU 40–50% train, 50–70% of bandwidth decode). 5 sanity-check against a known number.

INTERVIEW Q&A | TRAINING**Q305. Compute-optimal model for 10^{21} FLOPs?**

$C = 6ND$, $D = 20N \rightarrow C = 120N^2$, $N = \sqrt{10^{21}/120} \approx 2.9\text{B}$ params, $D \approx 58\text{B}$ tokens. If it will serve 10^{12} tokens, inference ($2N$ per token) dominates: train a smaller model on more tokens.

Q306. 70B on 15T tokens on 16,384 H100s at 40% MFU?

$6 \cdot 7 \cdot 10^{10} \cdot 1.5 \cdot 10^{13} = 6.3 \cdot 10^{24}$ FLOPs; cluster $16384 \cdot 989 \cdot 10^{12} \cdot 0.4 = 6.5 \cdot 10^{18}$ FLOP/s $\rightarrow 9.7 \cdot 10^5 \text{ s} \approx 11$ days (plus restarts, so plan 2–3 weeks).

Q307. Full FT of 7B on one 80 GB GPU? LoRA? QLoRA on 8 GB?

Full: $16 \times 7\text{B} = 112\text{GB}$, no (without 8-bit Adam / offload). LoRA on a bf16 base: 14 GB weights + small adapter state + activations, yes. QLoRA: $\approx 4\text{GB}$ weights, fits 8 GB only with short sequences, micro-batch 1 and checkpointing.

Q308. LoRA $r = 16$ on all linear layers of Llama-3-8B: trainable params?

Per layer $r(d_{\text{in}} + d_{\text{out}})$: q.o 16 · 8192; k.v 16 · 5120; gate, up, down 16 · 18432 $\rightarrow 1.31\text{M}$ per layer $\times 32 = 42\text{M} \approx 0.5\%$ of 8B. Adapter file $\approx 84\text{MB}$ in bf16.

Q309. How big is a 15T-token pretraining set on disk?

~ 4 bytes of text per token $\rightarrow \approx 60\text{TB}$ raw UTF-8. Tokenized: a 128k vocab needs 17 bits, so stored as uint32 $\rightarrow$ also $\approx 60\text{TB}$ (uint16 works only for vocab $\leq 65,536$). Compressed text is $\sim 3\text{--}4\times$ smaller.

Q310. Data-parallel all-reduce time per step for a 7B model on 400 Gb/s IB?

bf16 grads = 14 GB; ring sends $\approx 2 \times 14 = 28\text{GB}$ per GPU; at 50 GB/s $\approx 0.56\text{s}$. Must overlap with backward (bucketed all-reduce) or use FSDP/Zero with larger per-GPU batches to amortize.

INTERVIEW Q&A | SERVING**Q311. How many concurrent 8k-token requests fit for Llama-3-8B on one H100?**

KV per token = $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2\text{B} = 128\text{KiB}$. Memory for KV $\approx 72 - 16 = 56\text{GB}$ (90% util, bf16 weights) $\rightarrow 56 \cdot 10^9 / 131,072 \approx 427\text{k}$ tokens ≈ 52 sequences at 8k. FP8 KV doubles it.

Q312. Decode throughput, 8B bf16, batch 64, 2k context each?

Per step read weights 16 GB + KV 64 · 2048 · 128 KiB $\approx 17\text{GB} = 33\text{GB} \rightarrow \approx 10\text{ms}$ at 3.35 TB/s. Compute $2 \cdot 8 \cdot 10^9 \cdot 64 \approx 10^{12}$ FLOPs $\approx 1\text{ms}$: memory-bound. $\approx 6,400\text{ tok/s}$ total, $\sim 100\text{ tok/s}$ per user.

Q313. Batch-1 ceiling: 70B FP8 with TP=4?

17.5 GB per GPU / 3.35 TB/s $\approx 5.2\text{ms}$; 160 all-reduces at $\sim 10\mu\text{s} \approx 1.6\text{ms} \rightarrow \approx 7\text{ms/token}$, $\approx 140\text{ tok/s}$ before KV reads. TP=8 halves weight time but not comm.

Q314. Prefill 100k tokens on a 70B model, 8 H100 at 50%?

Weights $2 \cdot 7 \cdot 10^{10} \cdot 10^5 = 1.4 \cdot 10^{16}$; attention $2 \cdot L \cdot T^2 \cdot d = 2 \cdot 80 \cdot 10^{10} \cdot 8192 \approx 1.3 \cdot 10^{16}$; total $2.7 \cdot 10^{16}$ over $4 \cdot 10^{15}$ FLOP/s $\approx 7\text{s}$. KV = 320 KiB $\times 10^5 \approx 31\text{GiB}$.

Q315. Speculative decoding: $\alpha = 0.8$, $k = 4$, draft costs 5% of target per token. Speedup?

Tokens per round $(1 - \alpha^{k+1}) / (1 - \alpha) = (1 - 0.328) / 0.2 = 3.36$. Cost

per round $4 \cdot 0.05 + 1 = 1.2$ target steps $\rightarrow \approx 2.8\times$ (if verification of 5 tokens costs about one decode step, true when memory-bound).

Q316. Cost per 1M output tokens, 8B on an H100 at \$2/h, 5,000 tok/s? 18M tok/h $\rightarrow$ \$0.11/M at 100% utilization; at 30% average utilization \$0.37/M. Utilization is the number that moves the answer most.**Q317. GPUs for 1M DAU $\times$ 10 requests $\times$ 500 output tokens?**

$5 \cdot 10^9\text{ tok/day} = 58\text{k tok/s}$ average; peak $\approx 3\times \rightarrow 175\text{k tok/s}$; at 5k tok/s per GPU (8B, within latency SLO) ≈ 35 GPUs at peak, plus headroom and prefill capacity.

Q318. Vector index memory: 50M chunks, 768-d?

fp32 $50 \cdot 10^6 \cdot 768 \cdot 4 = 154\text{GB}$; fp16 77 GB; PQ 96 bytes/vector $\approx 4.8\text{GB}$ + HNSW links ($\sim 64\text{--}128\text{B/vec}$). Shard or quantize; rerank top-100 with full vectors.

INTERVIEW Q&A | EVALUATION**Q319. Model A 72%, B 70% on 500 items. Significant?**

SE per model $\sqrt{0.7 \cdot 0.3 / 500} \approx 2.0\text{pts}$; unpaired difference SE $\approx 2.9 \rightarrow 95\%$ CI ± 5.7 : no. Paired analysis (same items, McNemar or paired bootstrap) is tighter because per-item difficulty cancels; for a 2-point effect you typically need thousands of items.

Q320. How many items to detect a 3-point gain at 80% power?

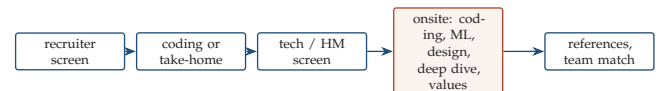
$n \approx \frac{(z_{\alpha/2} + z_{\beta})^2 \cdot 2p(1-p)}{\delta^2} = \frac{(1.96 + 0.84)^2 \cdot 0.42}{0.0009} \approx 3,660$ per arm unpaired; paired needs fewer depending on discordance.

Q321. Judge with 90% agreement on a task with 80% true pass rate. Bias?

If errors are symmetric (10% flips), observed pass rate = $0.8 \cdot 0.9 + 0.2 \cdot 0.1 = 0.74$: a 6-point underestimate. Correct with the confusion matrix estimated on human labels.

21 The interview loop: rounds, stories, DSA

[companies/ · career/stories.md · tracks/research-engineer · formats change, confirm with your recruiter]

**WHAT EACH LAB EMPHASIZES (REPORTED, VARIES BY TEAM AND YEAR)**

Anthropic	practical multi-part coding (CodeSignal/Colab, can look things up), ML/system design, project deep dive, values round strong engineers underestimate; no AI in live rounds; perf take-home is public
OpenAI	practical coding that grows in scope, in your editor; ML depth and debugging for research roles; LLM system design; every project number probed
Google / GDM	DSA medium in 20–25 min, often without running code; GDM “quiz” (rapid math/stats/ML breadth); ML rounds; hiring committee; JAX helps
Meta	2 DSA problems per 45 min; AI-enabled coding round in CoderPad (fix, implement, optimize); ML system design; behavioral by level (E4 vs E5 scope)
NVIDIA	C++/Python, CUDA execution model, memory hierarchy, roofline-style performance reasoning, DL fundamentals; outside AI tools disqualify

INTERVIEW Q&A | ROUND-BY-ROUND PLAYBOOK**Q322. Practical coding round: how to run it?**

Restate the spec, ask about scale and edge cases, write a tiny working version first, add tests as you go, narrate trade-offs, then extend. Clean names and small functions beat cleverness. Requirements will grow: design so step 2 is a small change.

Q323. ML coding (“implement attention / BPE / DPO”)?

State shapes before writing, write the core in 10 minutes, then test invariants (causality, equivalence to a library call, a gradient check). Mention dtype, masking and numerical stability unprompted (section 19).

Q324. ML / LLM system design structure?

Clarify goal and metric $\rightarrow$ constraints (QPS, latency, cost, privacy) $\rightarrow$ data $\rightarrow$ baseline $\rightarrow$ architecture diagram $\rightarrow$ evaluation of-line/online $\rightarrow$ serving and scaling $\rightarrow$ failure modes, monitoring, iteration. Quantify with napkin math at each step.

Q325. Project deep dive: what gets probed?

Data (test set, size, split, leakage); numbers (metric with CI, baseline tuned equally?); choices (why this loss/model, what failed); costs

(accuracy vs latency, on what hardware); next (10× compute, what would change your mind). Keep a numbers sheet per project.

Q326. Research-taste questions?

“Most surprising result you got”, “a paper you think is wrong”, “what would you work on with a cluster for a month”. Show a hypothesis, a cheap experiment to test it, and what result would falsify it.

Q327. Values / mission round?

Specific (named paper, policy, incident), personal (something you did), nuanced (state the strongest opposing view), calibrated (“I think X, I’d change my mind if Y”). Prepare one thing you’d refuse to build and one borderline case you’d accept with conditions.

S 20s	T 15s	A 60–90 s: your 3 decisions, alternatives, trade-offs	R numbers	L lesson
-------	-------	---	-----------	----------

STAR + learning, 2–3 minutes; “I” for your actions; every number traceable to a record

STORY BANK: 12–15 TRUE STORIES, EACH COVERING SEVERAL THEMES

hard technical problem	loss design, quantization trade-off, a bug you root-caused
ownership / ambiguity	a project you scoped from nothing
failure / mistake	badly scoped work and what you changed after
conflict / pushback	cost vs accuracy vs speed with a stakeholder
learning fast	a new domain in weeks; your from-scratch labs
leading without authority	contractors, cross-team asks
saying no	turning down bad-fit work
honesty under pressure	reporting a worse number than hoped
safety/privacy over speed	choosing on-prem/local for sensitive data
research taste	the most surprising experimental result

Follow-up drill, three levels deep: How did you measure it? What else did you consider? What if you’d done nothing? Who disagreed? What would you change? Where did you apply the lesson since?

THE 18 DSA PATTERNS AND THEIR TRIGGER SIGNALS

two pointers	sorted array pair/triplet, merge two sequences, palindrome
sliding window	longest/shortest substring or subarray with a condition
binary search	sorted, find target/boundary, need $O(\log n)$
search on answer	“smallest X such that Y ”, rotated array, peak values in $[1, N]$, find missing/duplicate
cyclic sort	reverse (part of) list, reorder, cycle, middle level by level, shortest path in tree, side views
linked-list reversal	path sums, construction, validation, LCA
tree BFS	components, shortest unweighted path, cycles, dependency order
graph BFS/DFS/topo	all subsets/permutations, N-queens, Sudoku
backtracking	interval scheduling, jump game, local choice provably optimal
greedy	running median, split into two groups
two heaps	k -th largest, k most frequent (size- k heap)
top- K	merge K sorted lists, k -th smallest in matrix
K -way merge	number of ways, min/max cost, overlapping subproblems
DP 1-D	two strings (LCS, edit distance), grids
DP 2-D	prefix matching, word search, autocomplete
trie	dynamic connectivity, counting components
union-find	

Standard: a medium in 20–25 min, narrated, with complexity and edge cases. Practise follow-ups: “doesn’t fit in memory”, “now it’s a stream”, “now concurrent”.

QUESTIONS TO ASK THEM, AND WHAT NOT TO ASK

HM: what success looks like at 6 months, the team’s hardest open problem, how research becomes product. Engineers: how experiments are reviewed, eval culture, on-call. Skip-level: strategy for two years. Never ask what the website already answers.

INTERVIEW Q&A | TRAINING

Q328. Loss is NaN at step 3,000.

Find the first non-finite tensor (anomaly mode or hooks on a replay of the batch). Usual causes: LR too high or no warmup, fp16 overflow (switch to bf16 or check the loss scaler), log(0) or division by zero in custom code, attention logits blowing up (QK-norm, z-loss), a corrupt batch. Rewind to the last good checkpoint, skip the batch, lower LR.

Q329. Loss plateaus way above expected from step 0.

Check initial loss $\approx \ln V$; overfit one batch (must go to ≈ 0); label shift by one; mask wrong; LR off by 10×; frozen params (requires_grad); optimizer given the wrong param list; data shuffled into noise (tokenizer mismatch).

Q330. GPU OOM after an innocent change.

Longer sequences or bigger micro-batch (activations scale with $B \cdot T$), a tensor kept in a list for logging (holds the graph: use .item() or .detach()), eval without no_grad, fragmentation (expandable_segments). Fix with checkpointing, smaller micro-batch + accumulation, FSDP, fused CE for big vocab.

Q331. Throughput dropped 30% after a library upgrade.

Profile both versions on the same batch; compare kernel lists (a fused kernel replaced by an unfused path, FlashAttention fallback to math SDPA, recompilations in torch.compile), NCCL settings, data-loader workers. Pin versions in the image.

Q332. Multi-node job hangs at startup or mid-run.

A rank waiting at a collective others never reached (rank-dependent control flow, unequal batch counts, an exception on one rank). Set NCCL_DEBUG=INFO, TORCH_NCCL_ASYNC_ERROR_HANDLING, timeouts; check py-spy dump on every rank; check the network fabric and a bad node.

Q333. Validation loss improves but downstream evals get worse.

Eval harness or prompt format changed; overfitting a narrow data mix; the eval was contaminated before and is now not; tokenizer/template mismatch at eval time; metric noise (check the CI).

INTERVIEW Q&A | POST-TRAINING AND SERVING

Q334. DPO: rewards margin rises, outputs get longer and worse.

Both chosen and rejected log-probs falling (likelihood displacement); length exploitation in preferences. Lower LR, raise β , add SFT loss on chosen (RPO), length-normalize or length-control the data, check that pairs are on-policy.

Q335. GRPO reward goes up, held-out accuracy flat.

Reward hacking (format rewards, verifier holes), training on all-correct/all-wrong groups (zero advantage), entropy collapse (clip-higher, temperature), train/test prompt overlap. Read 20 samples before reading any curve.

Q336. vLLM p99 latency spikes under load.

KV cache full → preemption and recompute; long prompts blocking decode (enable chunked prefill); too large max_num_seqs; prefix cache misses after a prompt change. Check queue time vs prefill vs decode in metrics.

Q337. Quantized model fine on benchmarks, worse for users.

Benchmarks miss long-context, multilingual and tool-use regressions; outlier channels hurt by INT4; KV cache quantized too. Eval on production traffic samples, keep sensitive layers in higher precision, try AWQ/GPTQ with a domain calibration set.

Q338. Outputs differ between batch sizes with greedy decoding.

Non-determinism from batch-dependent kernels (reduction order in matmuls/attention), FP8/bf16 rounding; expected unless you use batch-invariant kernels. Compare log-prob differences, not strings.

Q339. RAG answers are good offline, bad in production.

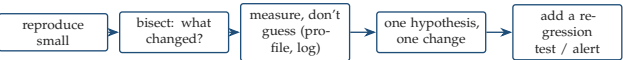
Offline set not representative (query distribution shift), stale index, tenant filters dropping docs, chunking differs between pipelines, prompt truncation at the context limit. Log retrieved ids per request and replay.

Q340. Agent loops forever calling the same tool.

Tool errors not surfaced in a useful form, no step budget, ambiguous tool descriptions, missing “done” criterion. Return actionable error messages, cap steps, dedupe identical calls, add a final-answer tool.

22 “What would you do?” debugging scenarios

[the round that separates people who have run things from people who have read about them]



23 Rapid fire: answer in one breath

[cover the right side, say the answer, check]

INTERVIEW Q&A | FUNDAMENTALS

Why log-softmax not log(softmax)? → numerical stability (LSE with max subtracted) + clean gradient $p - y$
Why does label smoothing help? → prevents over-confident logits; better calibration
Dropout in LLM pretraining? → usually 0: one epoch over huge data, little overfitting
Weight tying? → share embedding and LM head: saves Vd params; common in small models
Gradient clipping threshold? → global norm 1.0; log the pre-clip norm
Teacher forcing? → train on ground-truth prefixes; inference sees its own outputs (exposure bias)
Perplexity 20 in nats? → $\ln 20 \approx 3.0$
Why GELU/SiLU over ReLU? → smooth, non-zero gradient for negatives; empirically better
Vanishing gradients in transformers prevented by? → residuals + pre-norm + sane init
Causal vs bidirectional attention mask? → lower-triangular vs full
What is a logit lens? → decode intermediate residual streams with the final norm + unembedding
Gradient checkpointing cost? → $\approx 33\%$ compute (one extra forward)
Cross-entropy at uniform init? → $\ln V$
Embedding lookup FLOPs? → none (a gather); its backward is scatter-add

INTERVIEW Q&A | ARCHITECTURE

Params of a GPT block (4d GELU MLP, MHA)? → $12d^2$
GPT-2 small? → 124M: $d768, L12, H12, V50,257$, ctx 1024
Llama-3-8B? → $d4096, L32, H32/H_{kv}8, f14336, V128k$, RoPE $5 \cdot 10^5$
Llama-3-70B KV/token? → 320 KiB (80 layers, 8 KV heads)
Why RoPE over learned absolute? → relative by construction, no params, works with KV cache, extendable
ALiBi? → linear distance bias per head; extrapolates, recency-biased
Mixtral? → 8 experts, top-2, 46.7B total / 12.9B active
DeepSeek-V3? → 671B / 37B active, MLA, 256+1 experts top-8, MTP, FP8 training
Why no biases in LLMs? → fewer drifting params, simpler sharding/quant, no quality loss
Sliding window W over L layers reaches? → $L \cdot W$ tokens (not reliably)
Mamba vs attention, recall? → fixed state $\approx$ KV of 64 tokens; weak exact recall
QK-norm fixes? → attention-logit growth
Multi-token prediction? → extra heads/modules predict $t+2..$; denser signal; drafts for spec decoding
Parallel attention + MLP blocking (GPT-J, PaLM)? → $x + \text{Attn}(n(x)) + \text{MLP}(n(x))$: one fused matmul, $\sim 15\%$ faster

INTERVIEW Q&A | TRAINING AND SCALING

Chinchilla-optimal tokens for 7B? → $\approx 140B$ (20/param)
Training FLOPs, 7B on 2T tokens? → $6 \cdot 7 \cdot 10^9 \cdot 2 \cdot 10^{12} \approx 8.4 \cdot 10^{22}$
Days on 256 H100 at 40% MFU? → $8.4 \cdot 10^{22} / (256 \cdot 989 \cdot 10^{12} \cdot 0.4) \approx 9.6$ days
Memory for AdamW on 70B? → $16 \times 70B = 1.12$ TB before activations
ZeRO stage that shards weights? → 3 (FSDP full-shard)
Pipeline bubble formula? → $(p-1)/(m+p-1)$
Where does TP live? → inside a node, over NVLink
MoE comm primitive? → all-to-all
Why ramp batch size? → critical batch grows as loss falls
Loss spike remedy? → rewind ~ 100 steps, skip batches, lower LR; QK-norm, z-loss
 β_2 for LLMs? → 0.95
Weight decay applied to norms? → no (nor biases, often embeddings)
Peak LR for 7B AdamW? → $\sim 3e-4$ (confirm on a proxy / μP)
Repeated-data limit? → ~ 4 epochs nearly as good as fresh
FineWeb MinHash setup? → 5-grams, 112 hashes = 14 bands $\times$ 8 rows, ~ 0.75 Jaccard

INTERVIEW Q&A | POST-TRAINING

SFT loss on prompt tokens? → no; mask with -100 ; include end-of-turn
LoRA trainable params per matrix? → $r(d_{\text{in}} + d_{\text{out}})$
LoRA LR vs full FT? → $\sim 10 \times$ higher

QLoRA bits per param? → ≈ 4.5 (NF4 + fp32 scale per 64) → ~ 4.13 w/ double quant
DPO β default? → 0.1 (range 0.01–0.5); smaller lets policy move further
DPO implicit reward? → $\beta \log \pi_\theta(y|x) / \pi_{\text{ref}}(y|x)$
Why does $Z(x)$ vanish in DPO? → it depends only on x ; cancels in $r_w - r_l$
PPO clip ϵ ? → 0.2
PPO models in memory? → 4: policy, value, ref, reward
GRPO group size? → $G \approx 8-64$; advantage $(r - \bar{r}) / \sigma$
DAPO clip-higher? → $\epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$
Dr. GRPO removes? → per-sequence length norm and std normalization
KL estimator in GRPO? → $k_3 = \rho - 1 - \log \rho$
Reasoning RL on all-correct groups? → zero advantage: drop them (dynamic sampling)
RLHF KL direction? → reverse, $\text{KL}(\pi \| \pi_{\text{ref}})$
On-policy distillation reward? → per-token $-(\log q_\theta - \log p_{\text{teacher}})$

INTERVIEW Q&A | INFERENCE

Batch-1 decode bound? → BW / weight bytes (8B bf16 on H100 ≈ 200 tok/s)
Prefill intensity? → $\approx P$ (prompt length) $\Rightarrow$ compute-bound
Decode intensity? → $\approx B$ for weights, ≈ 1 (or g with GQA) for KV
vLLM block size? → 16 tokens
KV waste before/after paging? → 60–80% → $< 4\%$
Spec decode tokens per round? → $(1 - \alpha^{k+1}) / (1 - \alpha)$
Spec decode acceptance rule? → accept w.p. $\min(1, p/q)$; else sample $\text{norm}(\max(0, p - q))$
FP8 for inference? → e4m3 (max 448)
GPTQ uses? → inverse Hessian $H = 2X^\top X$ to push rounding error to later columns
AWQ protects? → $\sim 1\%$ salient channels by activation scale
SmoothQuant α ? → ≈ 0.5 ; migrates difficulty from activations to weights
INT4 g128 bits/weight? → 4.125
Online softmax state? → running max m , running sum ℓ , accumulator o
TTFT is dominated by? → queueing + prefill
Prefix cache killer? → dynamic content (timestamps) early in the prompt
Chunked prefill benefit? → no decode stalls; uses idle FLOPs
Min- p sampling? → keep tokens with $p \geq p_{\min} \cdot p_{\max}$

INTERVIEW Q&A | EVAL, RAG, AGENTS, SAFETY

95% CI, 1,000 items at 70%? → ± 2.8 points
Halve the CI? → $4 \times$ the items
pass@ k estimator? → $1 - \binom{n-\ell}{k} / \binom{n}{k}$
 κ with $p_o = .85, p_e = .68$? → 0.53
RRF constant? → $k = 60$
Most important retrieval metric for RAG? → $\text{recall}@k$
Contextual retrieval gain? → -49% failures; -67% with reranking
Lost in the middle? → mid-context info used worst; put key passages at ends
IVF nlist rule of thumb? → $\sim \sqrt{N}$ ($4-16\sqrt{N}$)
Agent 0.99/step, 50 steps? → 61%
pass⁸ at 90%? → 43%
Lethal trifecta? → private data + untrusted content + exfil channel
Refusal in open models mediated by? → one direction in the residual stream
Induction head pattern? → $\dots A B \dots A \rightarrow B$, via K-composition
SAE metrics? → L0, FVU, ΔCE , dead latents, auto-interp
FID measures? → distance between Inception-feature Gaussians, real vs generated
CFG formula? → $\epsilon_\emptyset + w(\epsilon_c - \epsilon_\emptyset)$

NAPKIN CONSTANTS

1 token	≈ 4 English characters ≈ 0.75 words
H100 SXM	989 TF/s bf16, 1,979 FP8 dense, 3.35 TB/s, 80 GB, NVLink 900 GB/s
H100 ridge	≈295 FLOP/byte (bf16)
A100 80G	312 TF/s, 2.0 TB/s
PCIe 5 x16	64 GB/s per direction
IB NDR	400 Gb/s = 50 GB/s
bytes/param	fp32 4, bf16 2, fp8 1, int4 0.5
train state	16 B/param (AdamW mixed precision)
KV (GQA-8, 128 dh)	$2 \cdot L \cdot 8 \cdot 128 \cdot 2 = 4L$ KiB/token
GPT-2 init loss	10.82 Llama 3: 11.76 Llama 2: 10.37
$\sqrt{128}$	11.3 $1/\sqrt{4096} = 1/64$
$e^{-1}, \ln 2$	0.368, 0.693

24 Formula sheet: write each from memory

[cover the page, rewrite, compare]

Core layers Scaled dot-product attention

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_h} + M)V, M_{ij} = -\infty \text{ for } j > i$$

Multi-head

$$\text{MHA}(x) = [h_1; \dots; h_H]W_O, h_i = \text{Attn}(xW_{Q_i}, xW_{K_i}^{g(i)}, xW_{V_i}^{g(i)}), g \text{ maps heads to KV groups}$$

RMSNorm / LayerNorm

$$\frac{x}{\sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}} \odot g \quad / \quad \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot g + b$$

SwiGLU MLP

$$(\text{SiLU}(xW_g) \odot xW_u)W_d, \text{SiLU}(z) = z \sigma(z)$$

RoPE (pair i , position m)

$$\theta_i = b^{-2i/d_h}; (x_{2i}, x_{2i+1}) \mapsto R(m\theta_i)(x_{2i}, x_{2i+1}); \langle R_m q, R_n k \rangle = f(q, k, m - n)$$

Pre-norm block

$$h = x + \text{Attn}(N(x)); \quad y = h + \text{MLP}(N(h))$$

Softmax-CE gradient

$$\partial L / \partial z = \text{softmax}(z) - e_y$$

Optimization AdamW

$$m \leftarrow \beta_1 m + (1 - \beta_1)g, v \leftarrow \beta_2 v + (1 - \beta_2)g^2, \theta \leftarrow \theta - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}} + \lambda \theta \right), \hat{m} = \frac{m}{1 - \beta_1^t}$$

Cosine schedule

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T)) \text{ after linear warmup}$$

Kaiming / Xavier

$$\text{Var}(W) = 2/n_{\text{in}} (\text{ReLU}) \quad / \quad 2/(n_{\text{in}} + n_{\text{out}})$$

Grad clipping

$$g \leftarrow g \cdot \min(1, c / \|g\|_2)$$

Scaling and compute FLOPs

forward $\approx 2N$ per token; training $C \approx 6ND$; attention adds $\approx 4LTd$ per token forward (causal halves it)

Chinchilla loss

$$L(N, D) = E + A/N^\alpha + B/D^\beta, \alpha \approx \beta \approx 0.3 \rightarrow N^* \propto C^{0.5}, D^* \approx 20N^*$$

Memory

train $\approx 16N$ bytes (bf16 W, G + fp32 master, m, v); KV = $2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ bytes/token

Roofline

attainable = $\min(\text{peak}, I \cdot \text{BW})$, I = FLOPs/byte; ridge = peak/BW

Pipeline bubble

$$(p-1)/(m+p-1); \text{ ring all-reduce bytes/GPU} = 2 \frac{n-1}{n} S$$

Post-training RLHF objective and optimum

$$\max_{\pi} \mathbb{E}[r] - \beta \text{KL}(\pi \| \pi_{\text{ref}}) \rightarrow \pi^* \propto \pi_{\text{ref}} e^{r/\beta}$$

Bradley-Terry RM loss

$$-\log \sigma(r(x, y_w) - r(x, y_l))$$

DPO

$$-\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right)$$

PPO clip

$$\mathbb{E}[\min(\rho A, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)A)], \rho = \pi_{\theta} / \pi_{\text{old}}$$

GAE

$$\delta_t = r_t + \gamma V_{t+1} - V_t, A_t = \sum_i (\gamma \lambda)^i \delta_{t+i}$$

GRPO advantage, KL

$$A_i = (r_i - \bar{r}) / \sigma_r \text{ over } G \text{ samples}; k_3 = \rho - 1 - \log \rho \text{ with } \rho = \pi_{\text{ref}} / \pi_{\theta}$$

Policy gradient

$$\nabla J = \mathbb{E}[\nabla \log \pi_{\theta}(y|x) (R - b)]$$

LoRA

$$W = W_0 + \frac{\alpha}{r} BA, B \in \mathbb{R}^{d \times r} = 0 \text{ at init}$$

Inference Sampling

$$p_i \propto e^{z_i/T}; \text{ top-}p: \text{ smallest set with } \sum p \geq p; \text{ min-}p: p_i \geq p_{\min} p_{\max}$$

Speculative decoding

$$\text{accept w.p. } \min(1, p/q); \text{ else sample } \propto \max(0, p - q); \mathbb{E}[\text{tokens}] = \frac{1 - a^{k+1}}{1 - a}$$

Online softmax

$$m' = \max(m, \max s), \ell' = \ell e^{m-m'} + \sum e^{s-m'}, o' = o e^{m-m'} + e^{s-m'} V$$

Quantization

$q = \text{clip}(\text{round}(x/s) + z), s = (\max - \min)/(2^b - 1)$ (asym) or $\max |x|/(2^{b-1} - 1)$ (sym)

Evaluation and retrieval Binomial CI

$$\hat{p} \pm 1.96 \sqrt{\hat{p}(1-\hat{p})/n}; \text{ sample size for effect } \delta: n \approx (z_{\alpha/2} + z_{\beta})^2 2p(1-p)/\delta^2$$

pass@k (unbiased)

$$1 - \binom{n-c}{k} / \binom{n}{k}$$

Cohen's κ

$$(p_o - p_e) / (1 - p_e)$$

InfoNCE

$$-\log \frac{e^{s(q,d^+)/\tau}}{\sum_j e^{s(q,d_j^+)/\tau}}$$

BM25

$$\sum_i \text{idf}(t) \frac{f(t,d)(k_1+1)}{f(t,d)+k_1(1-b+b|d|/|d|)}, k_1 \approx 1.2, b \approx 0.75$$

RRF, nDCG

$$\sum_r \frac{1}{60 + \text{rank}_r(d)}; \quad \text{DCG} = \sum_i \frac{2^{\text{rel}_i - 1}}{\log_2(i+1)}$$

Generative models DDPM

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon; \text{ loss } \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$$

Classifier-free guidance

$$\hat{\epsilon} = \epsilon_{\emptyset} + w(\epsilon_c - \epsilon_{\emptyset})$$

Flow matching

$$x_t = (1-t)x_0 + t\epsilon, \text{ regress } v_{\theta}(x_t, t) \approx \epsilon - x_0$$

VAE ELBO

$$\log p(x) \geq \mathbb{E}_q[\log p(x|z)] - \text{KL}(q(z|x) \| p(z))$$

25 Paper cards: what to say in 20 seconds

[curriculum/papers.md · claim, key number, limitation]

Attention Is All You Need (2017) Encoder-decoder of attention + MLP, no recurrence; parallel training; 28.4 BLEU EN-DE. *Limit:* quadratic in length; sinusoidal positions extrapolate poorly.

GPT-2 (2019) 1.5B decoder LM trained on WebText does tasks zero-shot; pre-norm. *Limit:* zero-shot far below fine-tuned SOTA.

GPT-3 (2020) 175B, few-shot in-context learning improves smoothly with scale. *Limit:* weak on reasoning; prompt sensitivity; not compute-optimal.

Kaplan scaling laws (2020) Loss is a power law in N, D, C ; bigger models are more sample-efficient. *Limit:* fixed LR schedule biased the N vs D split.

Chinchilla (2022) IsoFLOP sweeps: scale N and D equally; 70B/1.4T beats 280B Gopher. *Limit:* ignores inference cost; real models over-train.

InstructGPT (2022) SFT + RM + PPO; 1.3B preferred to 175B GPT-3 by raters. *Limit:* alignment tax; reward hacking; rater bias.

LoRA (2021) / QLoRA (2023) Low-rank adapters match full FT on many tasks with ~ 0.1 –1% params; QLoRA fine-tunes 65B on one 48 GB GPU via NF4. *Limit:* learns less and forgets less than full FT; weaker for new knowledge.

FlashAttention (2022) IO-aware tiling + online softmax: exact attention, $O(T)$ memory, 2 – $4\times$ faster. *Limit:* kernel engineering per hardware generation.

PagedAttention / vLLM (2023) KV in fixed blocks with block tables: near-zero fragmentation, 2 – $4\times$ throughput. *Limit:* indirection overhead in kernels.

Speculative decoding (2023) Draft + verify with rejection sampling: identical distribution, 2 – $3\times$ speedups. *Limit:* gains shrink at large batch (compute-bound).

DPO (2023) Closed-form RLHF optimum turns preference learning into a classification loss; no RM, no sampling. *Limit:* offline; likelihood displacement; sensitive to data distribution.

LLaMA (2023) Open 7–65B models trained past Chinchilla on public data; 13B rivals GPT-3. *Limit:* data and license constraints.

Mixtral (2023) 8 experts top-2: 47B total / 13B active matches Llama-2-70B. *Limit:* memory of all experts at serve time.

GPTQ (2022) / AWQ (2023) 3–4-bit weight-only quantization with small loss via Hessian compensation / salient-channel scaling. *Limit:* activation outliers still hard; calibration-set dependence.

Toy Models of Superposition (2022) Sparse features get packed into fewer dimensions; explains polysemanticity. *Limit:* toy setting.

In-context learning and induction heads (2022) Induction heads form in a phase change that coincides with ICL gains. *Limit:* causal evidence strongest in small models.

Lost in the Middle (2023) Models use information at the start/end of context best; U-shaped accuracy. *Limit:* newer long-context training reduces it.

RAG (2020) Retriever + generator trained jointly; better factuality on open QA. *Limit:* retrieval errors propagate.

DeepSeekMath / GRPO (2024) Group-relative advantages remove the value network; strong math RL. *Limit:* length bias, std normalization issues (Dr. GRPO).

DeepSeek-V3 (2024) 671B/37B MoE, MLA, aux-loss-free balancing, MTP, FP8 training for $\sim 2.8M$ H800-hours. *Limit:* complex to serve; reported cost excludes research.

DeepSeek-R1 (2025) RL on verifiable rewards yields long CoT reasoning; distilled small models are strong. *Limit:* readability, language mixing (R1-Zero); verifiable domains only.

Sparse autoencoders at scale (2024) Monosemantic, steerable features in production-size models. *Limit:* dead latents, absorption, reconstruction error.

Scaling test-time compute (2024) Compute-optimal mix of sampling and revision can beat a $14\times$ larger model on some problems. *Limit:* needs good verifiers; problem-difficulty dependent.

26 Glossary A–Z

ABF adjusted base frequency: raise RoPE base before long-context training.

Activation checkpointing store few activations, recompute in backward; +33% compute.

AdamW Adam with decoupled weight decay.

Alignment tax capability lost to safety/preference tuning.

All-reduce / all-gather / reduce-scatter / all-to-all the four collectives (section 3).

Attention sink first tokens absorb attention mass; keep them when evicting KV (StreamingLLM).

AWQ / GPTQ 4-bit weight quantization: salient-channel scaling / Hessian error compensation.

Beam search keep top- b partial sequences by log-prob; bland for chat.

BPE byte-pair encoding; merge most frequent pairs.

Bradley–Terry $P(a \succ b) = \sigma(r_a - r_b)$; RMs, DPO, arenas.

Capacity factor expert buffer size multiplier in MoE; overflow tokens dropped.

Chat template serialization of roles with special tokens; must match training.

Chinchilla 70B on 1.4T tokens; compute-optimal ≈ 20 tokens/param.

Chunked prefill split prompts into budgeted chunks interleaved with decode.

Constitutional AI self-critique against written principles + RLAI.

Context parallelism shard the sequence across GPUs for attention (ring).

Continuous batching batch re-formed every decode step.

Contrastive loss InfoNCE: pull positives together, push negatives apart.

Cross-encoder scores query and doc jointly; reranking.

DPO / IPO / KTO / SimPO / ORPO preference objectives without RL (section 9).

Distillation train a student on a teacher's outputs or logits.

Double descent test error falls again past the interpolation threshold.

Emergence abrupt task gains; often an artifact of discontinuous metrics.

Expert parallelism experts on different GPUs; all-to-all dispatch.

Exposure bias train on gold prefixes, generate on own prefixes.

FlashAttention IO-aware tiled exact attention with online softmax.

FSDP / ZeRO shard params, grads, optimizer states across data-parallel ranks.

Function calling model emits structured tool calls validated against JSON Schema.

GAE generalized advantage estimation, λ -weighted TD errors.

GQA / MQA / MLA fewer KV heads / one KV head / latent-compressed KV.

Grokking sudden generalization long after memorizing training data.

GRPO group-relative PPO without a value net (section 9).

Hallucination fluent unsupported content; measure faithfulness.

HNSW / IVF-PQ graph-based / clustered + compressed ANN indexes.

In-context learning learning from examples in the prompt, no weight update.

Induction head copy circuit $\dots A B \dots A \rightarrow B$.

Instruction hierarchy train to prioritize system > user > tool text.

KV cache stored keys/values of past tokens; $2LH_{kv}d_h$ elems/token.

Label smoothing target $(1 - \epsilon)$ on gold, ϵ spread elsewhere.

LoRA / QLoRA / DoRA low-rank adapters / on 4-bit base / magnitude-direction split.

Lost in the middle models use mid-context information worst.

Matryoshka embedding prefixes of the vector are valid embeddings.

MCP Model Context Protocol: standard tool/resource servers.

MFU / HFU useful FLOPs / all FLOPs incl. recompute, over peak.

Mixture of experts router picks top- k expert MLPs per token.

Model collapse degradation from recursive training on own synthetic data.

μ **P** parameterization making hyperparameters width-transferable.

Muon orthogonalized momentum for matrix params.

NF4 4-bit NormalFloat levels at normal quantiles.

Online softmax streaming softmax with running max and sum.

PagedAttention KV in fixed blocks via block tables.

pass@ k / pass k any of k succeeds / all k succeed.

Perplexity exp of mean NLL per token.

PPO clipped surrogate policy optimization.

Prefix caching reuse KV of shared prompt prefixes.

Prompt injection untrusted text hijacks model instructions.

PRM / ORM process vs outcome reward models.

QK-norm normalize q, k to stop logit growth.

RAG retrieve then generate with citations.

Reward hacking exploiting reward flaws instead of solving the task.

RLHF / RLAI / RLVR RL from human / AI feedback / verifiable rewards.

RMSNorm scale by root-mean-square, no centering.

RoPE / YaRN / NTK rotary positions and their extension tricks.

RRF reciprocal rank fusion, $\sum 1/(60 + \text{rank})$.

SAE sparse autoencoder dictionary over activations.

Self-consistency majority vote over sampled CoTs.

Speculative decoding draft k tokens, verify in one target pass, exact.

Superposition more features than dimensions, almost-orthogonal.

SwiGLU gated MLP with SiLU; hidden $\approx 8d/3$.

Sycophancy agreeing with the user over the truth.

Tensor parallelism split each matmul across GPUs (Megatron).

Test-time compute spend inference FLOPs (CoT, sampling, search) for accuracy.

Tied embeddings embedding matrix reused as LM head.

Top- k / top- p / min- p truncation sampling rules.

WSD warmup-stable-decay LR schedule.

z-loss penalty on $\log^2 Z$ to keep logits bounded.

27 References: the canon, by section

Math, optimization (1, 3) Parr & Howard, *The Matrix Calculus You Need for Deep Learning*. explained.ai

Kingma & Ba 2014, *Adam*. 1412.6980

Loshchilov & Hutter 2019, *Decoupled Weight Decay* (AdamW). 1711.05101

McCandlish et al. 2018, *Empirical Model of Large-Batch Training*. 1812.06162

Yang et al. 2022, *Tensor Programs V* (μ P). 2203.03466

Wortsman et al. 2023, *Small-scale proxies for large-scale instabilities*. 2309.14322

Hägele et al. 2024, *Scaling laws beyond fixed durations* (WSD). 2405.18392

Hardware (2) He, *Making Deep Learning Go Brrrrr*; DeepMind, *How to Scale Your Model*; HF *Ultra-Scale Playbook*. blogs

Korthikanti et al. 2022, *Reducing Activation Recomputation*. 2205.05198

Transformers, tokenizers (4, 5) Vaswani et al. 2017, *Attention Is All You Need*. 1706.03762

Sennrich et al. 2016, *BPE for NMT*; Kudo 2018, *SentencePiece*. 1508.07909, 1808.06226

Su et al. 2021, *RoFormer* (RoPE). 2104.09864

Shazeer 2019, MQA; Ainslie et al. 2023, GQA. 1911.02150, 2305.13245

Shazeer 2020, *GLU Variants*; Zhang & Sennrich 2019, RMSNorm. 2002.05202, 1910.07467

Xiong et al. 2020, *On LayerNorm in the Transformer* (pre-LN). 2002.04745

Chen et al. 2023, PI; Peng et al. 2023, YaRN. 2306.15595, 2309.00071

Fedus et al. 2021, *Switch Transformers*. 2101.03961

DeepSeek-V2 (MLA) and DeepSeek-V3 reports. 2405.04434, 2412.19437

Llama Team 2024, *The Llama 3 Herd of Models*. 2407.21783

Gu & Dao 2023, *Mamba*. 2312.00752

Pretraining (6) Kaplan et al. 2020; Hoffmann et al. 2022 (Chinchilla); Besiroglu et al. 2024 (replication). 2001.08361, 2203.15556, 2404.10102

Sardana et al. 2023, *Beyond Chinchilla-Optimal*. 2401.00448

Penedo et al. 2024, *FineWeb*; Li et al. 2024, *DCLM*. 2406.17557, 2406.11794

Lee et al. 2021, *Deduplicating Training Data*; Muennighoff et al. 2023, *Data-Constrained Scaling*. 2107.06499, 2305.16264

Rajbhandari et al. 2020, *ZeRO*; Zhao et al. 2023, *FSDP*. 1910.02054, 2304.11277

Shoeybi et al. 2019, Megatron-LM; Narayanan et al. 2021, PTD-P. 1909.08053, 2104.04473

Liu et al. 2023, *Ring Attention*. 2310.01889

Post-training (7) Ouyang et al. 2022, InstructGPT; Christiano et al. 2017, RLHF. 2203.02155, 1706.03741

Schulman et al. 2017, PPO; 2015, GAE. 1707.06347, 1506.02438

Rafailov et al. 2023, DPO; Azar et al. IPO; Ethayarajh et al. KTO; Meng et al. SimPO. 2305.18290, 2310.12036, 2402.01306, 2405.14734

Shao et al. 2023, DeepSeekMath (GRPO); DeepSeek-R1 2025. 2402.03300, 2501.12948

Yu et al. 2025, DAPQ; Liu et al. 2025, Dr.GRPO; Zheng et al. 2025, GSPO. 2503.14476, 2503.20783, 2507.18071

Hu et al. 2021, LoRA; Dettmers et al. 2023, QLoRA. 2106.09685, 2305.14314

Gao et al. 2022, *RM Overoptimization*; Bai et al. 2022, *Constitutional AI*. 2210.10760, 2212.08073

Lightman et al. 2023, *Let's Verify Step by Step*; Snell et al. 2024, test-time compute. 2305.20050, 2408.03314

Wei et al. 2022, CoT; Wang et al. 2022, self-consistency. 2201.11903, 2203.11171

Hinton et al. 2015, distillation; Agarwal et al. 2024, on-policy distillation (GKD). 1503.02531, 2306.13649

Inference (8) Pope et al. 2022, *Efficiently Scaling Transformer Inference*. 2211.05102

Dao et al. 2022 FlashAttention; Dao 2023 FA-2; Shah et al. 2024 FA-3. 2205.14135, 2307.08691, 2407.08608

Kwon et al. 2023, PagedAttention (vLLM); Zheng et al. 2024, SGLang. 2309.06180, 2312.07104

Agrawal et al. 2024, Sarathi-Serve; Zhong et al. 2024, DistServe. 2403.02310, 2401.09670

Leviathan et al. 2023; Chen et al. 2023, speculative sampling; Medusa; EAGLE. 2211.17192, 2302.01318, 2401.10774, 2401.15077

Frantar et al. GPTQ; Lin et al. AWQ; Xiao et al. SmoothQuant; Dettmers et al. LLM.int8(). 2210.17323, 2306.00978, 2211.10438, 2208.07339

Evaluation (9) Miller 2024, *Adding Error Bars to Evals*. 2411.00640

Chen et al. 2021, Codex / pass@ k ; Zheng et al. 2023, LLM-as-a-Judge. 2107.03374, 2306.05685

Biderman et al. 2024, *Reproducible Evaluation*; Oren et al. 2023, contamination. 2405.14782, 2310.17623

Yao et al. 2024, τ -bench; Jimenez et al. SWE-bench; Kwa et al. 2025, METR horizons. 2406.12045, 2310.06770, 2503.14499

Interpretability and safety (10) Elhage et al. 2021, *Mathematical Framework for Transformer Circuits*. transformer-circuits.pub

Olsson et al. 2022, *Induction Heads*; Elhage et al. 2022, *Toy Models of Superposition*. 2209.11895, 2209.10652

Bricken et al. 2023, *Towards Monosemanticity*; Gao et al. 2024, TopK SAEs. t-c.pub, 2406.04093

Wang et al. 2022, IOI circuit; Ardit et al. 2024, refusal direction. 2211.00593, 2406.11717

Greshake et al. 2023, indirect prompt injection; Hubinger et al. 2024, Sleeper Agents. 2302.12173, 2401.05566

Greenblatt et al. 2024, *Alignment Faking*; Greenblatt et al. 2023, *AI Control*. 2412.14093, 2312.06942

RAG, agents (11) Lewis et al. 2020, RAG; Karpukhin et al. 2020, DPR; Khattab & Zaharia 2020, ColBERT. 2005.11401, 2004.04906, 2004.12832

Malkov & Yashunin 2016, HNSW; Cormack et al. 2009, RRF. 1603.09320, SIGIR'09

Liu et al. 2023, *Lost in the Middle*; Gao et al. 2022, HyDE. 2307.03172, 2212.10496

Yao et al. 2022, ReAct; Shinn et al. 2023, Reflexion; Anthropic, *Building Effective Agents*. 2210.03629, 2303.11366

Kusupati et al. 2022, Matryoshka embeddings; Anthropic, *Contextual Retrieval*. 2205.13147

Generative models (12) Kingma & Welling 2013, VAE; Goodfellow et al. 2014, GAN. 1312.6114, 1406.2661

Ho et al. 2020, DDPM; Song et al. 2020, DDIM; Song et al. 2021, score SDEs. 2006.11239, 2010.02502, 2011.13456

Ho & Salimans 2022, classifier-free guidance; Rombach et al. 2022, latent diffusion. 2207.12598, 2112.10752

Peebles & Xie 2023, DiT; Lipman et al. 2023, flow matching; Esser et al. 2024, SD3. 2212.09748, 2210.02747, 2403.03206

Dosovitskiy et al. 2020, ViT; Radford et al. 2021, CLIP; Zhai et al. 2023, SigLIP. 2010.11929, 2103.00020, 2303.15343

Liu et al. 2023, LLaVA; Li et al. 2023, BLIP-2; Radford et al. 2022, Whisper. 2304.08485, 2301.12597, 2212.04356

Courses and books Karpathy, *Zero to Hero* + nanoGPT; Stanford CS336 (LMs from scratch), CS224N; Raschka, *Build a LLM from Scratch*; Jurafsky & Martin, *SLP3*; Bishop, *Deep Learning*; Chip Huyen, *AI Engineering*; Achilles labs 01–18.

28 Four-week plan and the cold self-check

[spaced repetition: read, recall out loud, re-test only the misses]

FOUR WEEKS TO "ANSWER IN MY SLEEP"		
week	read + recall (sections)	• build (Achilles labs)
1	math, foundations, compute, GPU, training, tokenizers	• labs 01–04 (autograd, training core, napkin math, BPE)
2	transformer, pretraining, post-training, reasoning	• labs 05, 08–12 (GPT, scaling, parallelism, LoRA, DPO, GRPO)
3	inference, eval, interp/safety	• labs 06, 07, 13–16 (kernels, KV cache, quantization, spec decoding, eval stats, interp)
4	RAG/agents, production, GenAI, design, code, napkin, loops	• labs 17–18 (retrieval, MoE); two mock interviews

Daily: 20 rapid-fire cards, one formula rewritten from memory, one napkin problem on paper, one from-scratch snippet typed blind. Re-test misses on days 3, 7, 21.

THE COLD SELF-CHECK: CAN YOU, WITHOUT NOTES, ...

- ☐ derive the softmax-CE gradient and the matmul VJP
- ☐ compute FLOPs, training memory and KV bytes for any model card
- ☐ place a kernel on the roofline and say what would move it
- ☐ compute occupancy and a GEMM tile’s arithmetic intensity
- ☐ explain pre-norm, RMSNorm, SwiGLU sizing and RoPE extension
- ☐ count Llama-3-8B’s parameters to within 1%
- ☐ explain GQA vs MLA and why GQA helps decode only
- ☐ write attention, RoPE, BPE and LoRA in 10 minutes each
- ☐ derive DPO from the RLHF objective in five steps
- ☐ say what GRPO removes from PPO and its three known bugs
- ☐ explain why speculative decoding is lossless and estimate its speedup
- ☐ size a serving fleet for given QPS, prompt and output lengths
- ☐ pick a quantization method for a given batch size and justify it
- ☐ give a CI for an eval score and say whether a 2-point gain is real
- ☐ design a RAG system and name where quality is lost first
- ☐ list prompt-injection defenses, strongest first
- ☐ run a 7-step system-design answer with numbers
- ☐ tell three STAR stories with sourced numbers, three follow-ups deep
- ☐ name the trigger signals of all 18 DSA patterns
- ☐ debug a NaN, an OOM and a hung multi-node job out loud